

Vampire (68080) Programming Guide

9 August 2021

Index

<u>Apollo Core 68080 Instructions</u>
<u>AMMX Instructions</u>
<u>Addressing Modes</u>
<u>Very Fast EA-Modes</u>
<u>Fast EA-Modes</u>
<u>Slow EA-Modes</u>
<u>AMMX</u>
<u>Detecting AMMX in AmigaOS</u>
<u>Enabling AMMX in AmigaOS</u>
<u>Introduction to AMMX</u>
<u>Instruction Words and Addressing Modes</u>
<u>Saga information</u>
<u>Joystick info</u>
<u>Graphics Modes</u>
<u>Registers</u>
<u>Quick Links</u>
Attachments
<u>Summary of Instructions</u>
<u>Instruction Timings</u>

Compiled by:
DARRIN

Apollo Core 68080 Instructions

Integer

[ADDA](#)
[ADDQ](#)
[CMPA](#)
[LEA](#)
[MOVEA](#)
[PEA](#)
[SUBA](#)
[SUBQ](#)

[ABCD](#)
[NBCD](#)
[PACK](#)
[SBCD](#)
[UNPK](#)

[CAS](#)
[CAS2](#)
[TAS](#)

[MOVEC](#)
[MOVES](#)
[RESET](#)
[RTE](#)

[ADD](#)
[ADDI](#)

[ADDQ](#)
[ADDX](#)
[CMP](#)
[CMP2](#)
[CMPI](#)
[CMPM](#)
[DIVS](#)
[DIVSL](#)
[DIVU](#)
[DIVUL](#)
[EXT](#)
[EXTB](#)
[MULS](#)
[MULU](#)
[NEG](#)
[NEGX](#)
[SUB](#)
[SUBI](#)
[SUBQ](#)
[SUBX](#)
[TST](#)

[BSET](#)
[BTST](#)
[BCHG](#)
[BCLR](#)
[BFSET](#)
[BFTST](#)

[BFCHG](#)
[BFCLR](#)
[BFEXTS](#)
[BFEXTU](#)
[BFFFO](#)
[BFINS](#)

[AND](#)
[ANDI](#)
[ANDN](#)
[EOR](#)
[EORI](#)
[NOT](#)
[OR](#)
[ORI](#)

[ASL](#)
[ASR](#)
[LSL](#)
[LSR](#)
[ROL](#)
[ROR](#)
[ROXL](#)
[ROXR](#)

[SCC](#)
[BRA](#)
[BCC](#)

[BRA](#)
[BSR](#)
[JMP](#)
[JSR](#)
[LINK](#)
[RTD](#)
[RTR](#)
[RTS](#)
[UNLK](#)

[CHK](#)
[CHK2](#)
[TRAP](#)
[Trapcc](#)
[TRAPV](#)

[CLR](#)
[EXG](#)
[MOVE](#)
[MOVE16](#)
[MOVEM](#)
[MOVEP](#)
[MOVEQ](#)
[MOVEX](#)
[PERM](#)
[SWAP](#)

[ILLEGAL](#)

NOP

[ANDI to CCR](#)
[ANDI to SR`](#)
[EORI to CCR](#)
[MOVE to CCR](#)
[MOVE from CCR](#)
[MOVE from SR](#)
[Move to SR](#)
[MOVE T/F USP](#)
[ORI to CCR](#)

Floating Point

[FABS](#)
[FADD](#)
[FDIV](#)
[FMOVE](#)
[FMUL](#)
[FNEG](#)
[FRESTORE](#)
[FSAVE](#)
[FSUB](#)

AMMX Instructions

[BANK](#)
[MINITERM](#)
[BFLYW](#)
[BSEL](#)
[C2P](#)
[LOAD](#)
[LOADi](#)
[LSLQ](#)
[LSRQ](#)
[PABS](#)
[PACKUSWB](#)

[PACK3216](#)
[PADDB](#)
[PADDUSB](#)
[PADDUSW](#)
[PADDW](#)
[PAND POR PEORP](#)
[ANDN](#)
[PAVGB](#)
[PCMPccB](#)
[PCMPccW](#)
[PMAXxB](#)

[PMAXxW](#)
[PMINxB](#)
[PMINxW](#)
[PMUL88](#)
[PMULA](#)
[PMULH](#)
[PMULL](#)
[PMULL88](#)
[PSUBB](#)
[PSUBUSB](#)
[PSUBUSW](#)

[PSUBW](#)
[STORE](#)
[STOREC](#)
[STOREi](#)
[STOREilm](#)
[STOREm](#)
[TRANSHI](#)
[TRANSLO](#)
[UNPACK1632](#)
[VPERM](#)

Press Ctrl and click the item to go directly to the page

Addressing Modes

Very fast EA-modes. Free, up many per Cycle

Data	Register Direct This addressing mode specifies that the operand is in one of the 8 Data registers . D0..D7
Attributes:	size BYTE, WORD, LONG, QUAD
Syntax:	Dn
Example:	

Address	Register Direct This addressing mode specifies that the operand is in one of the 8 Address registers . A0..A6 are normal registers. A7 is the Stack pointer Address registers will always updated in size of LONG.
Syntax:	An
Example:	

Immediate	Immediate value can be used as source
Syntax:	#
Example:	MOVE.L #\$12345678,D0

Fast EA-modes. Free, up to 1 per Cycle

Address (An)	Register Indirect. The operand is in a memory location which is specified by the given address register.
Syntax:	(An)
Example:	

Address (An)+	Register Indirect with Post-increment. The operand is in a memory location which is specified by the given address register. After the operand is fetched, the address in the specified register is incremented by the length of the operand (typically 1, 2, 4, or 8).
Syntax:	(An)+
Example:	

Address -(An)	Register Indirect with Pre-decrement. The operand is in a memory location which is specified by the given address register. Before the operand is fetched, the address in the specified register is decremented by the length of the operand (typically 1, 2, 4, or 8).
Syntax:	-(An)
Example:	

Address (d16,An)	Register Indirect with Displacement. The operand is in a memory location referenced by an effective address computed by adding the contents of the specified address register and a signed 16-bit displacement.
Syntax:	(d16,An)
Example:	

Address (d8,An,Rn*Scale)	Register Indirect with Index. The operand is in a memory location referenced by an effective address computed by adding together the contents of the specified address register, a second data or address register, and a signed 8-bit displacement. The value in the second register can either be a full long word or a sign-extended word value. This is specified by a .W or .L extension after the register. If no extension is used, the assembler assumes a word value. The Index register can optionally be multiplied with 1, 2, 4, or 8. As ADDRESS register can be used A0 .. A7. As INDEX register can be used A0 .. A7 or D0 .. D7
Syntax:	(d8,An, Rn), (d8,An, Rn.W *8), (d8,An, Rn.L *4)
Example:	

Address (bd,An,Xn*Scale)	Register Indirect with Index. The operand is in a memory location referenced by an effective address computed by adding together the contents of the specified address register, a second data or address register, and an optional displacement. The displacement could be none, or 16bit or 32bit wide. The value in the second register can either be a full long word or a sign-extended word value. This is
---------------------------------	--

	specified by a .W or .L extension after the register. If no extension is used, the assembler assumes a word value. The Index register can optionally be multiplied with 1, 2, 4, or 8. As ADDRESS register can be used A0 .. A7. As INDEX register can be used A0 .. A7 or D0 .. D7
Syntax:	(bd,An, Rn), (bd,An, Rn.W *8), (bd,An, Rn.L *4)
Example:	

PC-Relative	Indirect with Displacement. The operand is in a memory location referenced by an effective address computed by adding the PC and a signed 16-bit displacement.
Syntax:	(d16,PC)
Example:	

PC-Relative (d8,PC,Rn*Scale)	Indirect with Index. The operand is in a memory location referenced by an effective address computed by adding together the PC a second data or address register, and a signed 8-bit displacement. The value in the second register can either be a full long word or a sign-extended word value. This is specified by a .W or .L extension after the register. If no extension is used, the assembler assumes a word value. The Index register can optionally be multiplied with 1 / 2 / 4 or 8. As INDEX register can be used A0 .. A7 or D0 .. D7
Syntax:	(d8,PC, Rn), (d8,PC, Rn.W *8), (d8,PC, Rn.L *4)
Example:	

PC-Relative (bd,PC,Xn*Scale)	Indirect with Index. The operand is in a memory location referenced by an effective address computed by adding together the PC a second data or address register, and an optional displacement. The displacement can be none, or 16bit or 32bit wide. The value in the second register can either be a full long word or a sign-extended word value. This is specified by a .W or .L extension after the register. If no extension is used, the assembler assumes a word value. The Index register can optionally be multiplied with 1, 2, 4, or 8. As INDEX register can be used A0 .. A7 or D0 .. D7
Syntax:	(d32, PC, Rn), (d32, PC, Rn.W *8), (d32, PC, Rn.L *4)
Example:	

Absolute (Short)	Short Address. The operand is in a memory location specified by the given address. This mode is specified when an address is followed by a .W extension. The CPU will sign extended the 16bit offset to a full 32bit address. Therefore this mode can address memory from \$FFFF8000 to \$00000000 to \$00007FFFF
Syntax:	(xxx).W, (xxx)
Example:	

Absolute (Long)	Long Address. This mode is specified when an address is followed by a .L extension. This mode can address the full 32bit address space.
Syntax:	(xxx).L
Example:	

Slow EA-modes. These EA take 4 Cycle

Syntax:	
Example:	

Syntax:	
Example:	

Apollo Core 68080 Instructions

ABCD	Add Decimal with Extend. Adds the source operand to the destination operand along with the extend bit, and stores the result in the destination location. The addition is performed using binary coded decimal arithmetic. The operands, which are packed BCD numbers, can be addressed in two different ways: 1. Data register to data register (Data Register Direct). 2. Memory to memory (Pre-decrement). This operation is a byte operation only.	
Operation	Source(base 10) + Destination (base 10) -->; Destination	
Syntax:	ABCD Dy, Dx ABCD -(Ay), -(Ax)	
Attributes:	Size = (Byte)	
Condition Codes:	X Set the same as the carry bit. N Undefined. Z Cleared if the result is nonzero. Unchanged otherwise. V Undefined. C Set if a decimal carry was generated. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	Normally the Z condition code bit is set via programming before the start of an operation. This allows successful tests for zero results upon completion of multiple-precision operations.	

ADD	Add. Adds the source operand to the destination operand using binary addition, and stores the result in the destination location. The size of the operation may be specified as byte, word or long..	
Operation	Source + Destination -->; Destination Note: Add can use as source also AN. Only ADD/SUB/CMP/MOVE can use AN as source.	
Syntax:	ADD <ea>, Dn ADD Dn, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a carry is generated. Cleared otherwise.	
Address Modes:		
Example:	1) D1 = \$11111111 3) D1 = \$11111111 D2 = \$22222222 D2 = \$22222222 ADD.B D1,D2 ADD.L D1,D2 2) D1 = \$11111111 D2 = \$22222222 ADD.W D1,D2	Result: 1) D2 = \$11111133 2) D2 = \$11113333 3) D2 = \$33333333
Notes:		

ADDA	Address. Adds the source operand to the destination address register and stores the result in the address register. The size of the operation may be specified as word or long. Word size source operands are sign extended to 32-bit quantities prior to the addition.	
Operation	Source + Destination -->; Destination	
Syntax:	ADDA <ea>, An	
Attributes:	Size = (Word, Long)	
Condition	Not affected.	

Codes:		
Address Modes:		
Example:	A1= \$00001234 ADDA.W #8123,A1 Mind that with .W the source will get sign extended to 32bit \$00001234 +\$FFFF8123 ----- \$FFFF9357 ----- D0= \$00095000 A0= \$00081234 ADDA.L D0,A0	Result: A1 = \$FFFF9357 A0 = \$00116234
Notes:		

ADDI	Add Immediate. Adds the immediate data to the destination operand, and stores the result in the destination location. The size of the operation may be specified as byte, word, or long. The size of the immediate data matches the operation size.	
Operation	Immediate Data + Destination --> Destination	
Syntax:	ADDI #<data>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a carry is generated. Cleared otherwise.	
Address Modes:		
Example:	D1 = \$11111111 ADDI.W #\$8888,D1 ----- D1 = \$22221111 ADDI.W #\$FFFF,D1	Results D1 = \$11119999 D1 = \$22221110
Notes:		

ADDQ	Add Quick. Adds the immediate data 1 to 8 to the ADDRESS register Always the entire address register is updated.	
Operation	Immediate Data + Destination --> Destination	
Syntax:	ADDQ #<data>, An	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected	
Address Modes:		
Example:	A1= \$00001234 ADDQ.L #8,A1	Result: A1 = \$0000123C
Notes:	ADDQ is useful to increment pointers	

ADDX	Add with Extend. Adds the source operand to the destination operand along with the extend bit
-------------	--

	and stores the result in the destination location. The operands can be addressed in two ways: 1. Data register to data register. 2. Memory to memory (Pre-decrement mode) The size of the operation can be specified as byte, word or long.	
Operation		
Syntax:	ADDX Dy, Dx ADDX -(Ay), -(Ax)	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Cleared if the result is nonzero. Unchanged otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a carry is generated. Cleared otherwise. Note: Normally the Z condition code bit is set via programming before the start of an operation. This allows successful tests for zero results upon completion of multiple-precision operations.	
Address Modes:		
Example:		Result:
Notes:	ADDX and SUBX are useful for processing number longer than 32bit.	

AND	Logical AND. Performs an AND operation of the source operand with the destination operand and stores the result in the destination location. The size of the operation can be specified as byte, word, long or quad. The contents of an address register may not be used as an operand.	
Operation	Source Destination -->; Destination	
Syntax:	AND <EA>, DN AND DN, <EA>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:	Dn, An, #data, (An), (An)+, -(An), (d16,An), (d32,An), (d8,An,Xn), (d16,An,Xn), (d32,An,Xn), (d16,PC), (d32,PC), (d8,PC,Xn), (d16,PC,Xn), (xxx).w, (xxx).L, ([bd,An,Xn],od) & ([bd,An],Xn,od)	
Example:	D1 = \$12345678 D2 = \$0000FFFF AND.L D2,D1	Result: D1 = \$00005670
Notes:		

ANDI	Logical AND Immediate. Performs an AND operation on the immediate data and the destination operand, and stores the result in the destination location. The size of the operation may be specified as byte, word, or long. The size of the immediate data matches the operation size.	
Operation	Immediate Data \sL Destination -->; Destination	
Syntax:	ANDI #<data>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	

Address Modes:		
Example:	D1 = \$12345678 ANDI.L #\$000000F,D1	Result: D1 = \$00000008
Notes:		

ANDI to CCR	AND Immediate to Condition Code Register. Performs an AND operation with the condition codes and stores the result in the low-order byte of the status register.	
Operation	Source \sL CCR --> CCR	
Syntax:	ANDI #<data>, CCR	
Attributes:	Size = (Byte)	
Condition Codes:	X Cleared if bit 4 of immediate operand is zero. Unchanged otherwise. N Cleared if bit 3 of immediate operand is zero. Unchanged otherwise. Z Cleared if bit 2 of immediate operand is zero. Unchanged otherwise. V Cleared if bit 1 of immediate operand is zero. Unchanged otherwise. C Cleared if bit 0 of immediate operand is zero. Unchanged otherwise.	
Address Modes:		
Example:		Result:
Notes:		

ANDI to SR	AND Immediate to Status Register.	
Operation	If supervisor state then Source $\wedge$ SR $\sim$ SR else TRAP	
Syntax:	ANDI #<data>,SR	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

ANDN	Performs an AND operation of the INVERTED source operand with the destination operand and stores the result in the destination location. The size of the operation can be specified as byte, word, or long. The contents of an address register may not be used as an operand.	
Operation	Source Destination --> Destination	
Syntax:	ANDN <ea>, Dn ANDN Dn, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared.	

	C Always cleared.	
Address Modes:		
Example:	D1 = \$12345678 D2 = \$0000FFFF ANDN.L D2,D1	Result: D1 = \$12340008
Notes:		

ASL	Arithmetic Shift Left. Arithmetically shifts the bits of the operand to the left. The carry bit receives the last bit shifted out of the operand. The shift count for the shifting of a register may be specified in two different ways: 1. Immediate -- The shift count is specified in the instruction (shift range 1-8). 2. Register -- The shift count is the value in the data register specified in instruction modulo 64. The size of the operation can be specified as byte, word, or long. An operand in memory can be shifted one bit only, and the operand size is restricted to word. Bits shifted out of the high-order bit go to both the carry and the extend bits; zeros are shifted into the low-order bit. The overflow bit indicates if any sign changes occur during the shift.	
Operation	Destination Shifted by <count> -->; Destination	
Syntax:	ASL Dx, Dy ASL #<data>, Dy ASL <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set according to the last bit shifted out of the operand. Unaffected for a shift count of zero. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if the most significant bit is changed at any time during the shift operation. Cleared otherwise. C Set according to the last bit shifted out of the operand. Unaffected for a shift count of zero.	
Address Modes:		
Example:		Result:
Notes:		

ASR	Arithmetic Shift Right. Arithmetically shifts the bits of the operand to the right. The carry bit receives the last bit shifted out of the operand. The shift count for the shifting of a register may be specified in two different ways: 1. Immediate -- The shift count is specified in the instruction (shift range 1-8). 2. Register -- The shift count is the value in the data register specified in instruction modulo 64. The size of the operation can be specified as byte, word, or long. An operand in memory can be shifted one bit only, and the operand size is restricted to word. Bits shifted out of the low-order bit go to both the carry and the extend bits; the sign bit (MSB) is shifted into the high-order bit.	
Operation	Destination Shifted by <count> -->; Destination	
Syntax:	ASR Dx, Dy ASR #<data>, Dy ASR <ea>	
Attributes:	Size = (Byte, Word, Long)	

Condition Codes:	
Address Modes:	
Example:	Result:
Notes:	

BCC	Branch Conditionally. If the specified condition is true, program execution continues at location (PC) + displacement. The PC contains the address of the instruction word of the Bcc instruction plus two. The displacement is a two's complement integer that represents the relative distance in bytes from the current PC to the 16-bit displacement (the word immediately following the instruction) is used. If the 8-bit displacement field in the instruction word is all ones (\$FF), the 32-bit displacement (long word immediately following the instruction) is used. Condition code cc specifies one of the following conditions.	
Operation	If (condition true) then PC + d --> PC	
Syntax:	Bcc <label>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	CC carry clear 0100 /C CS carry set 0101 C EQ equal 0111 Z GE greater or equal 1100 N $\neq$ V + /N $\neq$ /V GT greater than 1110 N $\neq$ V $\neq$ /Z + /N $\neq$ /V $\neq$ /Z HI high 0010 /C $\neq$ /Z LE less or equal 1111 Z + N $\neq$ /V + /N $\neq$ V LS low or same 0011 C + Z LT less than 1101 N $\neq$ /V + /N $\neq$ V MI minus 1011 N NE not equal 0110 /Z PL plus 1010 /N VC overflow clear 1000 /V VS overflow set 1001 V	
Address Modes:		
Example:		Result:
Notes:		

BCHG	Test Bit and Change. Tests a bit in the destination operand and sets the Z condition code appropriately, then inverts the specified bit in the destination. When the destination is a data register, any of the 32 bits can be specified by the modulo 32-bit number. When the destination is a memory location, the operation is a byte operation, and the bit number is modulo 8. In all cases, bit zero refers to the least significant bit. The bit number for this operation may be specified in either of two ways: 1. Immediate -- The bit number is specified in a second word of the instruction. 2. Register -- The specified data register contains the bit number.	
Operation	$\sim$ (<number> of Destination) --> Z; $\sim$ (<number> of Destination) --> <bit number> of Destination	
Syntax:	BCHG Dn, <ea> BCHG #<data>, <ea>	
Attributes:	Size = (Byte, Long)	
Condition Codes:	X Not affected. N Not affected. Z Set if the bit tested is zero. Cleared otherwise. V Not affected. C Not affected.	

Address Modes:	
Example:	Result:
Notes:	

BCLR	Test Bit and Clear. Tests a bit in the destination operand and sets the Z condition code appropriately, then clears the specified bit in the destination. When the destination is a data register, any of the 32 bits can be specified by the modulo 32-bit number. When the destination is a memory location, the operation is a byte operation, and the bit number is modulo 8. In all cases, bit zero refers to the least significant bit. The bit number for this operation may be specified in either of two ways: 1. Immediate -- The bit number is specified in a second word of the instruction. 2. Register -- The specified data register contains the bit number.	
Operation	~(<number> of Destination) -->; Z; 0 -->; <bit number> of Destination	
Syntax:	BCLR Dn, <ea> BCLR #<data>, <ea>	
Attributes:	Size = (Byte, Long)	
Condition Codes:	X Not affected. N Not affected. Z Set if the bit tested is zero. Cleared otherwise. V Not affected. C Not affected.	
Address Modes:		
Example:		Result:
Notes:		

BFCHG	Test Bit Field and Change. Sets the condition codes according to the value in a bit field at the specified effective address, then complements the field. The field offset and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	~(<bit field> of Destination) -->; <bit field> of Destination	
Syntax:	BFCHG <ea>{offset:width}	
Attributes:	Unsize	
Condition Codes:	X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++--+++u6Effective Address,3Mode,3Reg,---1Do,5Offset,1Dw,5Width Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA78 Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31.	

	If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.
--	---

BFCLR	Test Bit Field and Clear. Sets the condition codes according to the value in a bit field at the specified effective address, and clears the field. The field offset and field width select field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	0 --> <bit field> of Destination	
Syntax:	BFCLR <ea>{offset:width}	
Attributes:	Unsize	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++---++u6Effective Address,3Mode,3Reg,---1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA78 Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31. If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.	

BFEXTS	Signed Bit Field Extract. Extracts a bit field from the specified effective address location, sign extends to 32 bits, and loads the result into the destination data register. The field offset
---------------	--

	and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	<bit field> of Source -->; Dn	
Syntax:	BFEXTS <ea>{offset:width}, Dn	
Attributes:	Unsize	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++--++++u6Effective Address,3Mode,3Reg,----1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA7B Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31. If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.	

BFEXTU	Unsigned Bit Field Extract. Extracts a bit field from the specified effective address location, zero extends to 32 bits, and loads the result into the destination data register. The field offset and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	<bit field> of Source -->; Dn	
Syntax:	BFEXTU <ea>{offset:width}, Dn	
Attributes:	Unsize	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:

Notes:	Instruction Format: \i2+++--+---+u6Effective Address,3Mode,3Reg,----1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA7B Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31. If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.
---------------	--

BFFFO	Bit Field Find First One. Searches the source operand for the most significant bit that is set to a value of one. The bit offset of that bit (the bit offset in the instruction plus the offset of the first one bit) is placed in Dn. If no bit in the bit field is set to one, the value in Dn is the field plus the field width. The instruction sets the condition codes according to the bit field value. The field offset and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	<bit field> of Source Bit Scan --> Dn	
Syntax:	BFFFO <ea>{offset:width}, Dn	
Attributes:	Unsize	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++--+---+u6Effective Address,3Mode,3Reg,----1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA7B Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31.	

	If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.
--	---

BFINS	Bit Field Insert. Inserts a bit field taken from the low-order bits of the specified data register into a bit field at the effective address location. The instruction sets the condition codes according to the inserted value. The field offset and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	Dn --> <bit field> of Destination	
Syntax:	BFINS Dn, <ea>{offset:width}	
Attributes:	Unsigned	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++-----u6Effective Address,3Mode,3Reg,---1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA78 Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31. If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.	

BKPT	Breakpoint
-------------	-------------------

Operation	Run breakpoint acknowledge cycle; TRAP as illegal instruction	
Syntax:	BKPT #<data>	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

BSEL	Performs the bitwise operation (A AND B) OR (NOT A AND C)	
Operation	Source Destination --> Destination	
Syntax:	BSEL <EA>, DN,DM	
Attributes:		
Condition Codes:	No Condition Codes affected.	
Address Modes:		
Example:	D1 = \$0000FFF000000FFF D2 = \$8899AABBCCDDEEFF D3 = \$0011223344556677 BSEL D1,D2,D3	Result: D3 = \$0011AAB344556EFF
Notes:		

BFSET	Test Bit Field and Set. Sets the condition codes according to the value in a bit field at the specified effective address, and sets the field. The field offset and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	1 --> <bit field> of Destination	
Syntax:	BFSET <ea>{offset:width}	
Attributes:	Unsize	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++---++u6Effective Address,3Mode,3Reg,----1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA78 Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31.	

	If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.
--	---

BFTST	Test Bit Field. Sets the condition codes according to the value in a bit field at the specified effective address. The field offset and field width select the field. The field offset specifies the starting bit of the field. The field width determines the number of bits in the field.	
Operation	<bit field> of Destination	
Syntax:	BFTST <ea>{offset:width}	
Attributes:	Unsigned	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the field is set. Cleared otherwise. Z Set if all bits in the field are zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2+++--++u6Effective Address,3Mode,3Reg,---1Do,5Offset,1Dw,5Width, Instruction Fields: Effective Address field -- Specifies the base location for the bit field. Only data register direct or control alterable addressing modes are allowed as shown: \mA7B Do field -- Determines how the field offset is specified. 0 -- The Offset field contains the bit field offset. 1 -- Bits [8:6] of the extension word specify a data register that contains the offset; bits [10:9] are zero. Offset field -- Specifies the field offset, depending on Do. If Do = 0, the Offset field is an immediate operand; the operand value is in the range 0-31. If Do = 1, the Offset field is a data register that contains the offset. The value is in the range -2^{31} to $2^{31} - 1$. Ds field -- Determines how the field width is specified. 0 -- The Width field contains the bit field width. 1 -- Bits [2:0] of the extension word specify a data register that contains the width; bits [3:4] are zero. Width field -- Specifies the field width, depending on Dw. If Dw = 0, the Width field is an immediate operand; an operand value is in the range 1-31 specifies a field width of 1-31, and a value of zero specifies a width of 32. If Dw = 1, the Width field is a data register that contains the width. The value modulo 32; values of 1-31 specify field widths of 1-31, and a value of zero specifies a width of 32.	

BRA	Branch. Program execution continues at location (PC) + displacement. The PC contains the
------------	---

	address of the instruction word of the BRA instruction plus two. The displacement is a twos complement integer that represents the relative distance in bytes from the current PC to the 16-bit displacement (the word immediately following the instruction) is used. If the 8-bit displacement field in the instruction word is all ones (\$FF), the 32-bit displacement (long word immediately following the instruction) is used. Condition code cc specifies one of the following conditions.	
Operation	If (condition true) then PC + d --> PC	
Syntax:	BRA <label>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i3-+-88-bit Displacement,016-bit Displacement if 8-bit Displacement = \$00,032-bit Displacement if 8-bit Displacement = \$FF, Instruction Fields (Register Shifts): Condition field -- The binary code for one of the conditions listed in the table. 8-bit Displacement field -- Twos complement integer specifying the number of bytes between the branch instruction and the next instruction to be executed if the condition is met. 16-bit Displacement field -- Used for the displacement when the 8-bit displacement field contains \$00. 32-bit Displacement field -- Used for the displacement when the 8-bit displacement field contains \$FF. Note: A branch to the immediately following instruction automatically uses the 16-bit displacement format because the 8-bit displacement field contains \$00 (zero offset). Related Instructions: Bcc,411 .422424	

BSET	Test Bit And Set. Tests a bit in the destination operand and sets the Z condition code appropriately, then sets the specified bit in the destination. When the destination is a data register, any of the 32 bits can be specified by the modulo 32-bit number. When the destination is a memory location, the operation is a byte operation, and the bit number is modulo 8. In all cases, bit zero refers to the least significant bit. The bit number for this operation may be specified in either of two ways: 1. Immediate -- The bit number is specified in a second word of the instruction. 2. Register -- The specified data register contains the bit number.	
Operation	~(<bit number> of Destination) --> Z; 1 --> <bit number> of Destination	
Syntax:	BSET Dn, <ea> BSET #<data>, <ea>	
Attributes:	Size = (Byte, Long)	
Condition Codes:	\c33233 X Not affected. N Not affected. Z Set if the bit tested is zero. Cleared otherwise. V Not affected. C Not affected.	
Address Modes:		
Example:		Result:
Notes:		

BSR	Branch To Subroutine. Pushes the long word address of the instruction immediately following the BSR instruction onto the system stack. The PC contains the address of the instruction plus two. Program execution continues at location (PC) + displacement. The PC contains the address of the instruction word of the BRA instruction plus two. The displacement is a two's complement integer that represents the relative distance in bytes from the current PC to the 16-bit displacement (the word immediately following the instruction) is used. If the 8-bit displacement field in the instruction word is all ones (\$FF), the 32-bit displacement (long word immediately following the instruction) is used. Condition code cc specifies one of the following conditions.	
Operation	SP-4 -->; SP; PC -->; (SP); PC + d -->; PC	
Syntax:	BSR <label>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

BTST	Test Bit. Tests a bit in the destination operand and sets the Z condition code appropriately. When the destination is a data register, any of the 32 bits can be specified by the modulo 32-bit number. When the destination is a memory location, the operation is a byte operation, and the bit number is modulo 8. In all cases, bit zero refers to the least significant bit. The bit number for this operation may be specified in either of two ways: 1. Immediate -- The bit number is specified in a second word of the instruction. 2. Register -- The specified data register contains the bit number.	
Operation	-(<bit number> of Destination) -->; Z;	
Syntax:	BTST Dn, <ea> BTST #<data>, <ea>	
Attributes:	Size = (Byte, Long)	
Condition Codes:	X Not affected. N Not affected. Z Set if the bit tested is zero. Cleared otherwise. V Not affected. C Not affected.	
Address Modes:		
Example:		Result:
Notes:		

CAS	Compare And Swap Operands. CAS compares the effective address operand to the compare operand (Dc). If the operands are equal, the instruction writes the update operand (Du) to the effective address operand; otherwise, the instruction writes the effective address operand to the compare operand (Dc). The instruction accesses memory using locked or read-modify-write transfer sequences. This provides a means of synchronizing several processors.	
Operation	CAS Destination -- Compare Operand -->; cc; if Z, Update Operand -->; Destination else Destination -->; Compare Operand	
Syntax:	CAS Dc, Du, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition	\c32222	

Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a borrow is generated. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	The CAS and CAS2 instructions can be used to perform secure update operations on system control data structures in a multiprocessing environment.	

CAS2	Compare And Swap Dual Operands. CAS2 compares memory operand 1 (Rn1) to compare operand 1 (Dc1). If the operands are equal, the instruction compares memory operand 2 (Rn2) to compare operand 2 (Dc2). If these operands are also equal, the instruction writes the update operands (Du1 and Du2) to the memory operands (Rn1 and Rn2). If either comparison fails, the instruction writes the memory operands (Rn1 and Rn2) to the compare operands (Dc1 and Dc2). The instruction accesses memory using locked or read-modify-write transfer sequences. This provides a means of synchronizing several processors.	
Operation	CAS2 Destination 1 -- Compare 1 -->; cc; if Z, Destination 2 -- Compare 2 -->; cc if Z, Update 1 -->; Destination 1; Update 2 -->; Destination 2 else Destination 1 -->; Compare 1; Destination 2 -->; Compare 2	
Syntax:	CAS2 Dc1:Dc2, Du1:Du2, (Rn1):(Rn2)	
Attributes:	Size = (Word, Long)	
Condition Codes:	\c32222 X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a borrow is generated. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	The CAS and CAS2 instructions can be used to perform secure update operations on system control data structures in a multiprocessing environment. In the MC68040 if the operands are not equal, the destination 1 operand is written back to memory to complete the locked access.	

CHK	Check Register Against Bounds. Compares the value in the data register specified in the instruction to zero and to the upper bound (effective address operand). The upper bound is a two's complement integer. If the register value is less than zero or greater than the upper bound, a CHK instruction exception, vector number 6, occurs.	
Operation	If Dn < 0 or Dn > Source then TRAP	
Syntax:	CHK <ea>, Dn	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected. N Set if Dn < 0; cleared if Dn > effective address operand. Undefined otherwise. Z Undefined. V Undefined. C Undefined.	
Address Modes:		
Example:		Result:
Notes:		

CHK2	Check Register Against Upper And Lower Bounds. Compares the value in Rn to each bound. The effective address contains the bounds pair: the lower bound followed by the upper bound. For signed comparisons, the arithmetically smaller value should be used as the lower bound. For unsigned comparisons, the logically smaller value should be the lower bound. The size of the data and the bounds can be specified as byte, word, or long. If Rn is a data register and operation size is a byte or word, only the appropriate low-order part of Rn is checked. If Rn is an address register and the operation size is byte or word, the bounds operands are sign extended to 32 bits and the resultant operands are compared to the full 32 bits of An. If the upper bound equals the lower bound, the valid range is a single value. If the register value is less than the lower bound or greater than the upper bound, a CHK instruction exception, vector number 6, occurs.	
Operation	If Rn < lower bound or Rn > upper bound then TRAP	
Syntax:	CHK2 <ea>, Rn	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	\c34242 X Not affected. N Undefined. Z Set if Rn is equal to either bound. Cleared otherwise. V Undefined. C Set if Rn is out of bounds. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	Instruction Format: \i2-----2Size,-++u6Effective Address,3Mode,3Reg,1DA,3Reg,+----- Instruction Fields: Size field -- Specifies the size of the operation. 00 -- Byte operation. 01 -- Word operation. 10 -- Long operation. Effective Address field -- Specifies the location of the bounds operands. Only control addressing modes are allowed as shown: \m27B DA field -- Specifies whether an address register or data register is to be checked. 0 -- Data register. 1 -- Address register. Register field -- Specifies the address or data register that contains the value to be checked. Related Instructions: CHK,430 .430432	

CLR	Clear. Clears the destination operand to zero. The size of the operation may be specified as byte, word, or long.	
Operation	0 -->; Destination	
Syntax:		
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	\c30100 X Not affected. N Always cleared. Z Always set. V Always cleared. C Always cleared.	
Address Modes:		

Example:	D0 = \$FEDCBA98 CLR.W D0	Result: D0 = \$DEDC0000
Notes:		

CMP	Compare. Subtracts the source operand from the destination data register and sets the condition codes according to the result; the data register is not changed. The size of the operation can be byte, word, or long.		
Operation	Destination - Source -->; cc		
Syntax:	CMP <ea>, Dn		
Attributes:	Size = (Byte, Word, Long)		
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow occurs. Cleared otherwise. C Set if a borrow occurs. Cleared otherwise.		
Address Modes:			
Example:			Result:
Notes:	CMP A,B signed unsigned ----- ----- ----- B <; A BLT BLO (or BCS) B <;= A BLE BLS B = A BEQ BEQ B <;> A BNE BNE B >; A BGT BHI B >;= A BGE BHS (or BCC) TST A signed unsigned ----- ----- ----- A <; 0 BLT (or BMI) NOP A <;= 0 BLE BZ (or BEQ) A = 0 BZ (or BEQ) BZ (or BEQ) A <;> 0 BNZ (or BNE) BNZ (or BNE) A >; 0 BGT BNZ (or BNE) A >;= 0 BGE (or BPL) BRA		

CMP2	Compare Register Against Upper And Lower Bounds. Compares the value in Rn to each bound. The effective address contains the bounds pair: the lower bound followed by the upper bound. For signed comparisons, the arithmetically smaller value should be used as the lower bound. For unsigned comparisons, the logically smaller value should be the lower bound. The size of the data and the bounds can be specified as byte, word, or long. If Rn is a data register and operation size is a byte or word, only the appropriate low-order part of Rn is checked. If Rn is an address register and the operation size is byte or word, the bounds operands are sign extended to 32 bits and the resultant operands are compared to the full 32 bits of An. If the upper bound equals the lower bound, the valid range is a single value.		
Operation	Compare Rn < lower bound or Rn > upper bound and Set Condition Codes		
Syntax:	CMP2 <ea>, Rn		
Attributes:	Size = (Byte, Word, Long)		
Condition Codes:	\c34242 X Not affected. N Undefined. Z Set if Rn is equal to either bound. Cleared otherwise. V Undefined. C Set if Rn is out of bounds. Cleared otherwise.		
Address			

Modes:		
Example:		Result:
Notes:	Instruction Format: \i2-----2Size,-++u6Effective Address,3Mode,3Reg,1DA,3Reg,----- Instruction Fields: Size field -- Specifies the size of the operation. 00 -- Byte operation. 01 -- Word operation. 10 -- Long operation. Effective Address field -- Specifies the location of the bounds operands. Only control addressing modes are allowed as shown: \m27B DA field -- Specifies whether an address register or data register is to be checked. 0 -- Data register. 1 -- Address register. Register field -- Specifies the address or data register that contains the value to be checked. Related Instructions: CMP,433 CMPA,434 CMPI,435 CMPM,436 .436438	

CMPA	Compare Address. Subtracts the source operand from the destination address register and sets the condition codes according to the result; the data register is not changed. The size of the operation can be word or long.	
Operation	Destination - Source -->; cc	
Syntax:	CMPA <ea>, An	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow occurs. Cleared otherwise. C Set if a borrow occurs. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	CMPA does the same operation as SUBA but does set the flags and does not update any registers. CMPA is the only instruction which works with ADDRESS registers which set any flags.	

CMPI	Compare Immediate. Subtracts the immediate data from the destination operand and sets the condition codes according to the result; the destination location is not changed. The size of the operation may be specified as byte, word, or long. The size of the immediate data matches the operation size.	
Operation	Destination - Immediate Data	
Syntax:	CMPI #<data>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow occurs. Cleared otherwise. C Set if a borrow occurs. Cleared otherwise.	
Address		

Modes:	
Example:	Result:
Notes:	

CMPM	Compare Memory to Memory. Subtracts the source operand from the destination operand and sets the condition codes according to the result; the destination location is not changed. The operands are always addressed with the post increment addressing mode, using the address registers specified in the instruction. The size of the operation may be specified as byte, word, or long.	
Operation	Compare 2 memory locations	
Syntax:	CMPM (Ay)+, (Ax)+	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow occurs. Cleared otherwise. C Set if a borrow occurs. Cleared otherwise.	
Address Modes:		
Example:	Im mem = \$1000 is the number \$1234 In mem = \$2000 is the number \$2345 A0 = \$1000 A1 = \$2000 CMPM.W (A0)+,(A1)+	Result: A0 and A1 will be incremented by two. The flags will be set accordingly. No memory content is changed.
Notes:		

DBCC	Test Condition, Decrement, and Branch.	
Operation	If condition false Then (Dn-1 ~ Dn; If Dn-1-1 then PC + dn ~ PC)	
Syntax:	DBcc Dn,<label>	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:	DBEQ decrement, branch equal. DBF decrement, branch false. (Same as DBRA.) DBGE decrement, branch greater than or equal. DBGT decrement, branch greater than. DBHI decrement, branch higher. DBLE decrement, branch less than or equal. DBLS decrement, branch less than or same. DBLT decrement, branch less than. DBMI decrement, branch minus. DBNE decrement, branch not equal. DBPL decrement, branch plus. DBRA decrement, branch unconditionally. DBT decrement, branch true. (Branch never taken.)	

DIVS	Single Divide. Divides the signed destination operand by the signed source operand and stores the signed result in the destination. The instruction uses one of three forms. The word form of the instruction divides a long word by a word. The result is a quotient in the lower word (least significant 16 bits) and the remainder is in the upper word (most significant 16 bits) of the result. The sign of the remainder is the same as the sign of the dividend. The first long form divides a long word by a long word. The result is a long quotient; the remainder is discarded. The second long form divides a quad word (in any two data registers) by a long word. The result is a long-word quotient and a long-word remainder. Two special conditions may arise during the operation: 1. Division by zero causes a trap. 2. Overflow may be detected and set before the instruction completes. If the instruction detects an overflow, it sets the overflow condition code and the operands are unaffected.	
Operation	Destination / Source --> Destination	
Syntax:	DIVS.W <ea>, Dn 32/16 --> 16r:16q DIVS.L <ea>, Dq 32/32 --> 32q DIVS.L <ea>, Dr:Dq 64/32 --> 32r:32q	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected. N Set if the quotient is negative. Cleared otherwise. Undefined if overflow or divide by zero occurs. Z Set if the quotient is zero. Cleared otherwise. Undefined if overflow or divide by zero occurs. V Set if the division overflow occurs; undefined if divide by zero occurs. Cleared otherwise. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

DIVSL	Single Divide. Divides the signed destination operand by the signed source operand and stores the signed result in the destination. This instruction divides a long word by a long word. The result is a long-word quotient and a long-word remainder. Two special condition may arise during the operation: 1. Division by zero causes a trap. 2. Overflow may be detected and set before the instruction completes. If the instruction detects an overflow, it sets the overflow condition code and the operands are unaffected.	
Operation	Destination / Source --> Destination	
Syntax:	DIVSL.L <ea>, Dr:Dq 32/32 --> 32r:32q	
Attributes:	Size = (Long)	
Condition Codes:	X Not affected. N Set if the quotient is negative. Cleared otherwise. Undefined if overflow or divide by zero occurs. Z Set if the quotient is zero. Cleared otherwise. Undefined if overflow or divide by zero occurs. V Set if the division overflow occurs; undefined if divide by zero occurs. Cleared otherwise. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

DIVU	Unsigned Divide. Divides the unsigned destination operand by the unsigned source operand	
-------------	---	--

	and stores the unsigned result in the destination. The instruction uses one of three forms. The word form of the instruction divides a long word by a word. The result is a quotient in the lower word (least significant 16 bits) and the remainder is in the upper word (most significant 16 bits) of the result. The first long form divides a long word by a long word. The result is a long quotient; the remainder is discarded. The second long form divides a quad word (in any two data registers) by a long word. The result is a long-word quotient and a long-word remainder. Two special condition may arise during the operation: 1. Division by zero causes a trap. 2. Overflow may be detected and set before the instruction completes. If the instruction detects an overflow, it sets the overflow condition code and the operands are unaffected.	
Operation	Destination / Source --> Destination	
Syntax:	DIVU.W <ea>, Dn 32/16 --> 16r:16q DIVU.L <ea>, Dq 32/32 --> 32q DIVU.L <ea>, Dr:Dq 64/32 --> 32r:32q	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected. N Set if the quotient is negative. Cleared otherwise. Undefined if overflow or divide by zero occurs. Z Set if the quotient is zero. Cleared otherwise. Undefined if overflow or divide by zero occurs. V Set if the division overflow occurs; undefined if divide by zero occurs. Cleared otherwise. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

DIVUL	Unsigned Divide. Divides the unsigned destination operand by the unsigned source operand and stores the unsigned result in the destination. This instruction divides a long word by a long word. The result is a long-word quotient and a long-word remainder. Two special condition may arise during the operation: 1. Division by zero causes a trap. 2. Overflow may be detected and set before the instruction completes. If the instruction detects an overflow, it sets the overflow condition code and the operands are unaffected.	
Operation	Destination / Source --> Destination	
Syntax:	DIVUL.L <ea>, Dr:Dq 32/32 --> 32r:32q	
Attributes:	Size = (Long)	
Condition Codes:	X Not affected. N Set if the quotient is negative. Cleared otherwise. Undefined if overflow or divide by zero occurs. Z Set if the quotient is zero. Cleared otherwise. Undefined if overflow or divide by zero occurs. V Set if the division overflow occurs; undefined if divide by zero occurs. Cleared otherwise. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

EOR	Logical Exclusive OR. Performs an exclusive-OR operation of the source operand with the destination operand and stores the result in the destination location. The size of the operation can be specified as byte, word, or long.	
------------	---	--

Operation	Source XOR Destination --> Destination	
Syntax:	EOR Dn, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:	D1 = \$F0008023 D2 = \$10008001 EOR.L D1,D2	Result: D2 = \$E0000022
Notes:		

EORI	Logical Exclusive OR Immediate. Performs an exclusive-OR operation on the immediate data and the destination operand, and stores the result in the destination location. The size of the operation may be specified as byte, word, or long. The size of the immediate data matches the operation size.	
Operation	Immediate Data \sÅ Destination --> Destination	
Syntax:	EORI #<data>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

EORI to CCR	Exclusive OR Immediate to the Condition Code Register. Performs an exclusive-OR operation with the condition codes and stores the result in the low-order byte of the status register.	
Operation	Source \sÅ CCR --> CCR	
Syntax:	EORI #<data>, CCR	
Attributes:	Size = (Byte)	
Condition Codes:	X Cleared if bit 4 of immediate operand is zero. Unchanged otherwise. N Cleared if bit 3 of immediate operand is zero. Unchanged otherwise. Z Cleared if bit 2 of immediate operand is zero. Unchanged otherwise. V Cleared if bit 1 of immediate operand is zero. Unchanged otherwise. C Cleared if bit 0 of immediate operand is zero. Unchanged otherwise.	
Address Modes:		
Example:		Result:
Notes:		

EXG	Exchange Registers. Exchanged the contents of two 32-bit registers. The instruction performs three types of exchanges: 1. Exchange data registers.	
------------	---	--

	2. Exchange address registers. 3. Exchange a data register and an address register.	
Operation	Rx ←--> Ry	
Syntax:	EXG Dx, Dy EXG As, Ay EXG Dx, Ay EXG Ay, Dx	
Attributes:	Size = (Quad)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

EXT	Sign Extend. Extends a byte in a data register to a word or a word in a data register to a long word, by replicating the sign bit to the left. If the operation extends a byte to a word, bit [7] of the designated data register is copied to bits [15:8] of that data register. If the operation extends a word to a long word, bit [15] of the designated data register is copied to bits [31:16] of the data register.	
Operation	Destination Sign-Extended --> Destination	
Syntax:	EXT.W Dn extends byte to word EXT.L Dn extends word to long word	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

EXTB	Sign Extend. Extends a byte in a data register to a long word by replicating the sign bit to the left. Bit [7] of the designated data register is copied to bits [31:16] of that data register.	
Operation	Destination Sign-Extended --> Destination	
Syntax:	EXTB.L Dn extends byte to long word	
Attributes:	Size = (Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

FABS	Floating-point Absolute Value	
Operation	Absolute Value of Source ~ FPn	
Syntax:	FABS.<fmt> <ea>,FPn FABS.X FPM,FPn	

	FABS.X FPn FrABS.<fmt> <ea>,FPn3 FrABS.X FPm,FPn3 FrABS.X FPn3	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

FADD	Floating-point Add	
Operation	Source + FPn ~ FPn	
Syntax:	FADD.<fmt> <ea>,FPn FADD.X FPm,FPn FrADD.<fmt> <ea>,FPn3 FrADD.X FPm,FPn3	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

FDIV	Floating-point Divide	
Operation	FPn , Source ~ FPn	
Syntax:	FDIV.<fmt> <ea>,FPn FDIV.X FPm,FPn FrDIV.<fmt> <ea>,FPn3 FrDIV.X FPm,FPn3	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

FMOVE	Move Floating-point Register	
Operation	Source ~ Destination	
Syntax:	FMOVE.<fmt> <ea>,FPn FMOVE.<fmt> FPm,<ea> FMOVE.P FPm,<ea>{Dn} FMOVE.P FPm,<ea>{#k} FrMOVE.<fmt> <ea>,FPn3	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:

Notes:	
---------------	--

FMUL	Floating-point Multiply	
Operation	Source $\times$ FPn $\rightarrow$ FPn	
Syntax:	FMUL.<fmt> <ea>,FPn FMUL.X FPm,FPn FrMUL.<fmt> <ea>,FPn3 FrMUL.X FPm,FPn3	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

FNEG	Floating-point Negate	
Operation	-(Source) $\rightarrow$ FPn	
Syntax:	FNEG.<fmt> <ea>,FPn FNEG.X FPm,FPn FNEG.X FPn FrNEG.<fmt> <ea>,FPn3 FrNEG.X FPm,FPn3 FrNEG.X FPn3	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

FRESTORE	Restore Floating-point Internal State	
Operation	If in supervisor state then FPU State Frame $\rightarrow$ Internal State else TRAP	
Syntax:	FRESTORE <ea>	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

FSAVE	Save Floating-point Internal State	
Operation	If in supervisor state then FPU Internal State $\rightarrow$ State Frame else TRAP	
Syntax:	FSAVE <ea>	
Attributes:		
Condition		

Codes:	
Address Modes:	
Example:	Result:
Notes:	

FSUB	Floating-point Subtract
Operation	FPn – Source ~ FPn
Syntax:	FSUB.<fmt> <ea>,FPn FSUB.X FPM,FPn FrSUB.<fmt> <ea>,FPn3 FrSUB.X FPM,FPn3
Attributes:	
Condition Codes:	
Address Modes:	
Example:	Result:
Notes:	

ILLEGAL	Take Illegal Instruction Trap. Forces an illegal instruction exception, vector number 4. All other illegal instruction bit patterns are reserved for future extension of the instruction set and should not be used to force an exception.
Operation	*SSP - 2 -->; SSP; Vector Offset -->; (SSP); SSP - 4 -->; SSP; PC -->; (SSP); SSP - 2 -->; SSP; SR-->; (SSP); Illegal Instruction Vector Address -->; PC
Syntax:	ILLEGAL
Attributes:	Unsize
Condition Codes:	Not Affected

JMP	Jump. Program execution continues at the effective address specified by the instruction.
Operation	Destination Address -->; PC
Syntax:	JMP <ea>
Attributes:	Unsize
Condition Codes:	Not affected.
Address Modes:	
Example:	Result:
Notes:	

JSR	Jump to Subroutine. Pushes the long-word address of the instruction immediately following the JSR instruction onto the system stack. Program execution then continues at the effective address specified by the instruction.
Operation	SP - 4 -->; SP; PC -->; (SP); Destination Address -->; PC
Syntax:	JSR <ea>
Attributes:	Unsize
Condition Codes:	Not affected.

Address Modes:		
Example:		Result:
Notes:		

LEA	Load Effective Address. Loads the effective address into the specified address register. All 32 bits of the address register are affected by this instruction.	
Operation	<ea> -->; An	
Syntax:	LEA <ea>,An	
Attributes:	Size = (Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:	A1= \$00011004 LEA (\$2000,A1),A2	Result: A2 = \$00013004
Notes:		

LINK	Link And Allocate. Pushes the contents of the specified address register onto the stack. Then loads the updated stack pointer into the address register. Finally, adds the displacement value to the stack pointer. For word-size operation, the displacement is the sign-extended word following the operation word. For long-size operation, the displacement is the long word following the operation word. The address register occupies one long word on the stack. The user should specify a negative displacement in order to allocate	
Operation	SP - 4 -->; SP; An -->; (SP); SP -->; An; SP + d -->; SP	
Syntax:	LINK An, #<displacement>	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

LSL	Logical Shift Left. Shifts the bits of the operand to the left. The carry bit receives the last bit shifted out of the operand. The shift count for the shifting of a register may be specified in two different ways: 1. Immediate -- The shift count is specified in the instruction (shift range 1-8). 2. Register -- The shift count is the value in the data register specified in instruction modulo 64 The size of the operation can be specified as byte, word, or long. An operand in memory can be shifted one bit only, and the operand size is restricted to word. Bits shifted out of the high order bit go to both the carry and the extend bits; zeros are shifted into the high-order bit.	
Operation	Destination Shifted by <count> -->; Destination	
Syntax:	LSL Dx, Dy LSL #<data>, Dy LSL <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set according to the last bit shifted out of the operand. Unaffected for a shift count of zero. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise.	

	V Always cleared. C Set according to the last bit shifted out of the operand. Cleared for a shift count of zero.	
Address Modes:		
Example:		Result:
Notes:		

LSR	Logical Shift Right. Shifts the bits of the operand to the right. The carry bit receives the last bit shifted out of the operand. The shift count for the shifting of a register may be specified in two different ways: 1. Immediate -- The shift count is specified in the instruction (shift range 1-8). 2. Register -- The shift count is the value in the data register specified in instruction modulo 64 The size of the operation can be specified as byte, word, or long. An operand in memory can be shifted one bit only, and the operand size is restricted to word. Bits shifted out of the low order bit go to both the carry and the extend bits; the sign bit (MSB) is shifted into the high order bit.	
Operation	Destination Shifted by <count> -->; Destination-	
Syntax:	LSR Dx, Dy LSR #<data>, Dy LSR <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set according to the last bit shifted out of the operand. Unaffected for a shift count of zero. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Set according to the last bit shifted out of the operand. Cleared for a shift count of zero.	
Address Modes:		
Example:		Result:
Notes:		

Move from CCR	Move Condition Code Register. Moves the condition code bits (zero extended to word size) to the destination location. The operand size is a word. Unimplemented bits are read as zeros.	
Operation	CCR -->; Destination	
Syntax:	MOVE CCR, <ea>	
Attributes:		
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

Move to CCR	Move Condition Code Register	
Operation	Source ~ CCR	
Syntax:	MOVE <ea>,CCR	
Attributes:		
Condition		

Codes:	
Address Modes:	
Example:	Result:
Notes:	

Move from SR	Move Status Register . Moves the data in the status register to the destination location. The destination is word length. Unimplemented bits are read as zeros.	
Operation	SR --> Destination	
Syntax:	MOVE SR, <ea>	
Attributes:	Size = (Word)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

Move to SR	Move Status Register	
Operation	If supervisor state then Source ~ SR else TRAP	
Syntax:	MOVE <ea>,SR	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

Move to/from USP	Move User Stack Pointer	
Operation	If supervisor state Then USP ~ An or An ~ USP else TRAP	
Syntax:	MOVE USP,An MOVE An,USP	
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

ORI to CCR		
Operation	Source V CCR ~ CCR	
Syntax:	ORI #<data>,CCR	
Attributes:		
Condition		

Codes:	
Address Modes:	
Example:	Result:
Notes:	

MOVE	Move. Moves the data at the source to the destination location, and sets the condition codes according to the data. The size of the operation may be specified as byte, word, long.	
Operation	Source -->; Destination	
Syntax:	MOVE <ea>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	When the source operand uses a direct Address Register addressing mode, the assembler converts it to a MOVEA. \b	

MOVE16	16-byte Block Move . Moves the source line to the destination line. Applications for this instruction include coprocessor communications, memory initialization, and fast block copy operations. MOVE16 has two formats. The post increment format uses the post increment addressing mode for both source and destination, while the absolute format specifies an absolute long address for either the source or destination. Line transfers are performed using burst reads and writes which begin with the long word pointed to by the <ea> of the source and destination, respectively. An address register used in the post increment addressing mode is incremented by 16 after the transfer.	
Operation	Source Block -->; Destination Block	
Syntax:	MOVE16 (Ax)+, (Ay)+ MOVE16 xxx.L, (An) MOVE16 xxx.L, (An)+ MOVE16 (An), xxx.L MOVE16 (An)+, xxx.L	
Attributes:	Size = (16 Byte)	
Condition Codes:	Not affected.	
Address Modes:		
Example:	MOVE16 (A0), \$FE802 A0 = \$1400F The line at address \$14000 is read into a temporary holding register by a burst read transfer starting with long word \$1400C.	Result: Address values in A0 or \$14000-\$1400F cause the same line to be read, starting at different long words. The line is then written to the line at address \$FE800 beginning with long word \$FE800. After the instruction A0 contains \$1401F.
Notes:		

MOVEA	Move Address. Moves the data at the source to the destination location, and sets the condition codes according to the data. The size of the operation may be specified as byte, word, or long.
--------------	---

Operation	Source -->; Destination	
Syntax:	MOVEA <ea>, AN	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:	A0= \$00008000 A1= \$00044444 MOVE.L A0,A1 ----- A0= \$00008000 A1= \$00044444 MOVE.W A0,A1 ; Mind .W gets sign extended to 32bit	Result: A1 = \$00008000 A1 = \$FFFF8000
Notes:		

MOVEC	Move Control Register . Move General Purpose Register to Control Register. Or move control register to general purpose register.	
Operation	Source -->; Destination	
Syntax:	MOVEC CR, DN	
Attributes:	Size = (LONG)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

MOVEM	Move Peripheral . Moves the contents of selected register to or from consecutive memory locations starting at the location specified by the effective address. A register is selected if the bit in the mask field corresponding to that register is set. The instruction size determines whether 16 or 32 bits of each register are transferred. In the case of a word transfer to either address or data registers, each word is sign extended to 32 bits, and the resulting long word is loaded into the associated register. Selecting the address mode also selects the mode of operation of the MOVEM instruction, and only the control modes, the pre-decrement mode, and the post-increment mode are valid. If the effective address is specified by one of the control modes, the registers are transferred starting at the specified address, and the address is incremented by the operand length (2 or 4) following each transfer. The order of the registers is from data register 0 to data register 7, then from address register 0 to address register 7. If the effective address is specified by the pre-decrement mode, only a register-to-memory operation is allowed. The registers are stored starting at the specified address minus the operand length (2 or 4), and the address is decremented by the operand length following each transfer. The order of storing is from address register 7 to address register 0, then from data register 7 to data register 0. When the instruction has completed, the decremented address register contains the address of the last operand stored. If the effective address is specified by the post-increment mode, only a memory-to-register operation is allowed. The registers are loaded starting at the specified address; the address is incremented by the operand length (2 or 4) following each transfer. The order of loading is the same as that of control mode addressing. When the instruction has completed, the incremented address register contains the address of the last operand plus the operand length. If the addressing register is also loaded from memory, the memory value is ignored and the register is
--------------	--

	written with the post-incremented effective address.	
Operation	Registers -->; Destination Source -->; Registers	
Syntax:	MOVEM register list, <ea> MOVEM <ea>, register list	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:	Register List Mask field -- Specifies the register to be transferred. The low-order bit corresponds to the first register to be transferred; the high-order bit corresponds to the last register to be transferred. Thus, both for control modes and for the post-increment mode addresses, the mask correspondence is: A7,A6,A5,A4,A3,A2,A1,A0,D7,D6,D5,D4,D3,D2,D1,D0 For the predecrement mode addresses, the mask correspondence is reversed: D0,D1,D2,D3,D4,D5,D6,D7,A0,A1,A2,A3,A4,A5,A6,A7 Register Lists: The assembler syntax for register lists is as follows: 1. A range of registers is specified with a hyphen (minus sign) as in A0-A3. Note that ranges must begin with the smaller of the two registers. All data registers are considered smaller than all address registers, so D7-A0 is a legal range. 2. A set of distinct registers is specified by a list separated by slashes (division signs) as in D3/D7/A0. These may be in any order. 3. A combination of 1 and 2, such as A0-A3/D3/D6-D7 may be specified. The assembler just ignores registers which are specified more than once, as in A3/A2-A4	

MOVEP	Move Quick . Moves data between a data register and alternate bytes within the address space starting at the location specified and incrementing by two. The high-order byte of the data register is transferred first and the low-order byte is transferred last. The memory address is specified in the address register indirect plus 16-bit displacement addressing mode. This instruction was originally designed for interfacing 8-bit peripherals on a 16-bit data bus, such as the MC68000 bus.	
Operation	Source -->; Destination	
Syntax:	MOVEP Dx, d(Ay) MOVEP d(Ay), Dx	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

MOVEQ	Move Alternate Address Space . Moves a byte of immediate data to a 32-bit data register. The data in an 8-bit field within the operation word is sign extended to a long operand in the data register as it is transferred.	
Operation	Immediate Data -->; Destination	
Syntax:		
Attributes:		
Condition	\c32200	

Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:	MOVEQ #\$80,D0 ----- MOVEQ #\$77,D0	Result: D0 = \$FFFFFF80 ----- D0 = \$00000077
Notes:		

MOVES	Move Multiply . This instruction moves from a general purpose register to the address space specified by the DFC.	
Operation	Source -->; Destination	
Syntax:	MOVES <ea>, RN MOVES Rn,<ea>	
Attributes:	Size = (BYTE, WORD, LONG)	
Condition Codes:	X Not affected.	
Address Modes:		
Example:		Result:
Notes:		

MOVEX	This instruction moves from a general purpose register to the EA and converts the data to Little Endian. Or it reads from EA , Converts from Little Endian to Big Endian and updates the Register.	
Operation	Source -->; Destination	
Syntax:	MOVEX <ea>, RN MOVEX Rn,<ea>	
Attributes:	Size = (WORD, LONG)	
Condition Codes:	X Not affected.	
Address Modes:		
Example:		Result:
Notes:		

MULS	Signed Multiply . Multiplies two signed operands yielding a signed result. This instruction has a word operand form and a long-word operand form. In the word form, the multiplier and multiplicand are both word operands, and the result is a long-word operand. A register operand is the low-order word; the upper word of the register is ignored. All 32 bits of the product are saved in the destination data register. In the long form, the multiplier and multiplicand are both long-word operands, and the result is either a long word or a quad word. The long-word result is the low-order 32 bits of the quad word result; the high-order 32 bits of the product are discarded.	
Operation	Source * Destination -->; Destination	
Syntax:	MULS.W <ea>, Dn 16 x 16 -->; 32 MULS.L <ea>, D1 32 x 32 -->; 32 MULS.L <ea>, Dh:D1 32 x 32 -->; 64	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected.	
Codes:	N Set if the result is negative. Cleared otherwise.	

	Z Set if the result is zero. Cleared otherwise. V Set if overflow. Cleared otherwise. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Overflow (V = 1) can occur only when multiplying 32-bit operands to yield a 32-bit result. Overflow occurs if the high-order 32 bits of the quad-word product are not the sign extension of the low-order 32 bits.	

MULU	Unsigned Multiply Multiplies two unsigned operands yielding an unsigned result. This instruction has a word operand form and a long-word operand form. In the word form, the multiplier and multiplicand are both word operands, and the result is a long-word operand. A register operand is the low-order word; the upper word of the register is ignored. All 32 bits of the product are saved in the destination data register. In the long form, the multiplier and multiplicand are both long-word operands, and the result is either a long word or a quad word. The long-word result is the low-order 32 bits of the quad word result; the high-order 32 bits of the product are discarded.	
Operation	Source * Destination --> Destination	
Syntax:	MULU.W <ea>, Dn 16 x 16 --> 32 MULU.L <ea>, D1 32 x 32 --> 32 MULU.L <ea>, Dh:D1 32 x 32 --> 64	
Attributes:	Size = (Word, Long)	
Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if overflow. Cleared otherwise. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:	Overflow (V = 1) can occur only when multiplying 32-bit operands to yield a 32-bit result. Overflow occurs if the high-order 32 bits of the quad-word product are not the sign extension of the low-order 32 bits.	

NBCD	Negate decimal with extend . Subtracts the source operand and the extend bit from zero. The operation is performed using binary coded decimal arithmetic. The packed BCD result is saved in the destination location. This instruction produces the tens complement of the destination if the extend bit is zero, or the nines complement if the extend bit is one. This operation is a byte operation only.	
Operation	0 - Destination (base 10) - X --> Destination	
Syntax:	NBCD <ea>	
Attributes:	Size = (Byte)	
Condition Codes:	\c24242 X Set the same as the carry bit. N Undefined. Z Cleared if the result is nonzero. Unchanged otherwise. V Undefined. C Set if a decimal carry was generated. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	Normally the Z condition code bit is set via programming before the start of an operation. This allows successful tests for zero results upon completion of multiple-precision operations.	

NEG	Negate. Subtracts the destination operand from zero and stores the result in the destination location. The size of the operation is specified as byte, word, or long.	
Operation	0 - (Destination) --> Destination	
Syntax:	NEG <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow occurs. Cleared otherwise. C Cleared if the result is zero. Set otherwise.	
Address Modes:		
Example:		Result:
Notes:	NEG is used to change a positive number into a negative. If the number to change is longer than 32bit then use NEGX.	

NEGX	Negate with extend . Subtracts the destination operand and the extend bit from zero and stores the result in the destination location. The size of the operation is specified as byte, word, or long.	
Operation	0 - (Destination) - X --> Destination	
Syntax:	NEGX <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow occurs. Cleared otherwise. C Cleared if the result is zero. Set otherwise.	
Address Modes:		
Example:		Result:
Notes:	Normally the Z condition code bit is set via programming before the start of the operation. This allows successful tests for zero results upon completion of multiple precision operations. NEGX can be used to negate numbers longer than 32bit	

NOP	No Operation. Performs no operation. The processor state, other than the program counter, is unaffected. Execution continues with the instruction following the NOP instruction. The NOP instruction does not begin execution until all pending bus cycles are completed. This synchronizes the pipeline, and prevents instruction overlap.	
Operation	None	
Syntax:	NOP	
Attributes:	Unsize	
Condition Codes:	Not Affected	

NOT	Logical Complement. Calculates the ones complement of the destination operand and stores the result in the destination location. The size of the operation may be specified as byte, word, or long.	
Operation	~Destination --> Destination	
Syntax:	NOT <ea>	
Attributes:	Size = (Byte, Word, Long)	

Condition Codes:	X Not affected. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

OR	Logical Inclusive OR . Performs an inclusive-OR operation of the source operand with the destination operand and stores the result in the destination location. The size of the operation can be specified as byte, word, or long. The contents of an address register may not be used as an operand.	
Operation	Source V Destination -->; Destination	
Syntax:	OR <ea>, Dn OR Dn, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:	D1 = \$FF00200F D2 = \$0100800F OR.L D1,D2	Result: D2 = \$FF00A00F
Notes:		

ORI	Logical Inclusive OR Immediate . Performs an inclusive-OR operation on the immediate data and the destination operand, and stores the result in the destination location. The size of the operation may be specified as byte, word, or long. The size of the immediate data matches the operation size.	
Operation	Immediate Data V Destination -->; Destination	
Syntax:	ORI #<data>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:	D1 = \$FF00200F ORI.L #\$123,D1	Result: D1 = \$FF00A12F
Notes:		

PACK	Pack BCD . Adjusts and packs the low four bits of each of two bytes into a single byte.	
Operation	Source (Unpacked BCD) + adjustment -->; Destination (Packed BCD)	

Syntax:	PACK -(Ax), -(Ay), #<adjustment> PACK Dx, Dy, #<adjustment>	
Attributes:	Unsize	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:	When both operands are data registers, the adjustment is added to the value contained in the source register. Bits [11:8] and [3:0] of the intermediate result are concatenated and placed in bits [7:0] of the destination register. The remainder of the destination register is unaffected.	

PEA	Push Effective Address . Computes the effective address and pushes it onto the stack. The effective address is a long-word address.	
Operation	SP - 4 -->; SP; <ea> -->; (SP)	
Syntax:	PEA <ea>	
Attributes:	Size = (Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:	A1= \$22222222 D0= \$12345678 PEA (5,A1,D0)	Result: On the stack the value: \$3456789F will be pushed. D0 and A1 are unchanged.
Notes:		

PERM	Performs a byte permutation of two data registers to create one 32bit result.	
Operation	Source Destination -->; Destination	
Syntax:	PERM <#imm>, Dn,Dm	
Attributes:	Size = LONG	
Condition Codes:	Not affected.	
Address Modes:		
Example:	D1 = \$01234567 D2 = \$89ABCDEF PERM #0167,D1:D2	Result: D3 = \$0123CDEF
Notes:		

PPERM		
Operation		
Syntax:		
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

RESET	Reset External Devices
--------------	-------------------------------

Operation		
Syntax:		
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

ROL	Rotate Left . Rotates the bits of the operand to the left. The extend bit is not included in the rotation. The rotate count for the rotation of a register is specified in either of two ways: 1. Immediate -- The rotate count (1-8) is specified in the instruction. 2. Register -- The rotate count is the value in the data register specified in the instruction, modulo 64. The size of the operation for register destinations is specified as byte, word, or long. The contents of memory, <ea>, can be rotated one bit only, and operand size is restricted to a word. The ROL instruction rotates the bits of the operand to the left; the rotate count determines the number of bit positions rotated. Bits rotated out of the high order bit go to the carry bit and also back into the low-order bit.	
Operation	Destination Rotated by <count> -->; Destination	
Syntax:	ROL Dx, Dy ROL #<data>, Dy ROL* <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Set according to the last bit rotated out of the operand. Cleared when the rotate count is zero.	
Address Modes:		
Example:		Result:
Notes:		

ROR	Rotate Right . Rotates the bits of the operand to the right. The extend bit is not included in the rotation. The rotate count for the rotation of a register is specified in either of two ways: 1. Immediate -- The rotate count (1-8) is specified in the instruction. 2. Register -- The rotate count is the value in the data register specified in the instruction, modulo 64. The size of the operation for register destinations is specified as byte, word, or long. The contents of memory, <ea>, can be rotated one bit only, and operand size is restricted to a word. The ROR instruction rotates the bits of the operand to the right; the rotate count determines the number of bit positions rotated. Bits rotated out of the low order bit go to the carry bit and also back into the high-order bit.	
Operation	Destination Rotated by <count> -->; Destination	
Syntax:	ROR Dx, Dy ROR #<data>, Dy ROR* <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the most significant bit of the result is set.	

	Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Set according to the last bit rotated out of the operand. Cleared when the rotate count is zero.	
Address Modes:		
Example:		Result:
Notes:		

ROXL	Rotate With Extend Left .Rotates the bits of the operand to the left. The extend bit is included in the rotation. The rotate count for the rotation of a register is specified in either of two ways: 1. Immediate -- The rotate count (1-8) is specified in the instruction. 2. Register -- The rotate count is the value in the data register specified in the instruction, modulo 64. The size of the operation for register destinations is specified as byte, word, or long. The contents of memory, <ea>, can be rotated one bit only, and operand size is restricted to a word. The ROXL instruction rotates the bits of the operand to the left; the rotate count determines the number of bit positions rotated. Bits rotated out of the high order bit go to the carry bit and the extend bit; the previous value of the extend bit rotates into the low-order bit.	
Operation	Destination Rotated with X by <count> -->; Destination	
Syntax:	ROXL Dx, Dy ROXL #<data>, Dy ROXL* <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set according to the last bit rotated out of the operand. Cleared when the rotate count is zero. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Set according to the last bit rotated out of the operand. Cleared when the rotate count is zero.	
Address Modes:		
Example:		Result:
Notes:		

ROXR	Rotate With Extend Right . Rotates the bits of the operand to the right. The extend bit is included in the rotation. The rotate count for the rotation of a register is specified in either of two ways: 1. Immediate -- The rotate count (1-8) is specified in the instruction. 2. Register -- The rotate count is the value in the data register specified in the instruction, modulo 64. The size of the operation for register destinations is specified as byte, word, or long. The contents of memory, <ea>, can be rotated one bit only, and operand size is restricted to a word. The ROXR instruction rotates the bits of the operand to the right; the rotate count determines the number of bit positions rotated. Bits rotated out of the low order bit go to the carry bit and the extend bit; the previous value of the extend bit rotates into the high-order bit.	
Operation	Destination Rotated with X by <count> -->; Destination	
Syntax:	ROXR Dx, Dy ROXR #<data>, Dy ROXR* <ea>	

Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set according to the last bit rotated out of the operand. Cleared when the rotate count is zero. N Set if the most significant bit of the result is set. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Always cleared. C Set according to the last bit rotated out of the operand. Cleared when the rotate count is zero.	
Address Modes:		
Example:		Result:
Notes:		

RPC	Return Program Counter	
Operation		
Syntax:		
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:	The RPC points to the location where fetching will commence after transition from background mode to normal mode. This register should be accessed to change the flow of a program under development. Changing the RPC to an odd value will cause an address error when normal mode prefetching begins.	

RTD	Return And Deallocate . Pulls the program counter value from the stack and adds the sign-extended 16-bit displacement value to the stack pointer. The previous program counter value is lost.	
Operation	Operation: (SP) -->; PC; SP + 4 + d -->; SP	
Syntax:	RTD #<displacement>	
Attributes:	Not Sized	
Condition Codes:	Not affected	
Address Modes:		
Example:		Result:
Notes:		

RTE	Return From Exception	
Operation	If supervisor state then (SP) ~ SR; SP + 2 ~ SP; (SP) ~ PC; SP + 4 ~ SP; restore state and deallocate Stack according to (SP) else TRAP	
Syntax:	RTE	
Attributes:		
Condition Codes:		
Address Modes:		

Example:		Result:
Notes:		

RTR	Return And Restore Codes . Pulls the condition code and program counter values from the stack. The previous condition codes and program counter values are lost. The supervisor portion of the status register is unaffected.	
Operation	(SP) --> CCR; SP + 2 --> SP; (SP) --> PC; SP + 4 --> SP	
Syntax:	RTR	
Attributes:	Unsize	
Condition Codes:	Set according to the condition codes from the stack.	
Address Modes:		
Example:		Result:
Notes:		

RTS	Return From Subroutine . Pops the program counter value from the stack. The previous program counter value is lost.	
Operation	(SP) --> PC; SP + 4 --> SP	
Syntax:	RTS	
Attributes:	Unsize	
Condition Codes:	Not affected	
Address Modes:		
Example:		Result:
Notes:		

SBCD	Subtract Decimal With Extend . Subtracts the source operand from the destination operand and stores the result in the destination location. The subtraction is performed using binary coded decimal arithmetic. The operands, which are packed BCD numbers, can be addressed in two different ways: 1. Data register to data register (Data Register Direct). 2. Memory to memory (Pre-decrement). This operation is a byte operation only.	
Operation	Destination (base 10) - Source(base 10) --> Destination	
Syntax:	ABCD Dx, Dy ABCD -(Ax), -(Ay)	
Attributes:	Size = (Byte)	
Condition Codes:	\c24242 X Set the same as the carry bit. N Undefined. Z Cleared if the result is nonzero. Unchanged otherwise. V Undefined. C Set if a decimal carry was generated. Cleared otherwise.	
Address Modes:		
Example:		Result:
Notes:	Normally the Z condition code bit is set via programming before the start of an operation. This allows successful tests for zero results upon completion of multiple-precision operations.	

SCC	Set Conditionally . Tests the specified condition code; if the condition is true, sets the byte
------------	--

	specified by the effective address to TRUE (all ones). Otherwise, sets that byte to FALSE (all zeros). Condition code cc specifies one of the following conditions. CC carry clear 0100 /C CS carry set 0101 C EQ equal 0111 Z F never equal 0001 0 GE greater or equal 1100 $N \neq V + /N \neq /V$ GT greater than 1110 $N \neq V \neq /Z + /N \neq /V \neq /Z$ HI high 0010 /C $\neq /Z$ LE less or equal 1111 $Z + N \neq /V + /N \neq V$ LS low or same 0011 C + Z LT less than 1101 $N \neq /V + /N \neq V$ MI minus 1011 N NE not equal 0110 /Z PL plus 1010 /N T always true 0000 1 VC overflow clear 1000 /V VS overflow set 1001 V	
Operation	If (condition true) then 1s -->; Destination else 0s -->; Destination	
Syntax:	Scc <label>	
Attributes:	Size = (Byte)	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:	A subsequent NEG.B instruction with the same effective address can be used to change the Scc result from TRUE or FALSE to the equivalent arithmetic value (TRUE=1, FALSE=0).	

SHIFT		
Operation		
Syntax:		
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

SHIFTL		
Operation		
Syntax:		
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

SHIFTR	
Operation	
Syntax:	
Attributes:	
Condition Codes:	
Address Modes:	
Example:	Result:
Notes:	

STOP	Stop
Operation	If supervisor state then Immediate Data ~ SR; STOP else TRAP
Syntax:	STOP #<data>
Attributes:	
Condition Codes:	Set according to the immediate operand.
Address Modes:	
Example:	Result:
Notes:	Moves the immediate operand into the status register (both user and supervisor portions), advances the program counter to point to the next instruction, and stops the fetching and executing of instructions. A trace, interrupt, or reset exception causes the processor to resume instruction execution. A trace exception occurs if instruction tracing is enabled (T0 = 1, T1=0) when the STOP instruction begins execution. If an interrupt request is asserted with a priority higher than the priority level set by the new status register value, an interrupt exception occurs; otherwise, the interrupt request is ignored. External reset always initiates reset exception processing.

SUB	Subtract . Subtracts the source operand from the destination operand and stores the result in the destination location. The size of the operation may be specified as byte, word, long.
Operation	Destination - Source --> Destination
Syntax:	SUB <ea>, Dn SUB Dn, <ea>
Attributes:	Size = (Byte, Word, Long)
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a carry is generated. Cleared otherwise.
Address Modes:	
Example:	D1 = \$11111111 D2 = \$22222222 SUB.B D2,D1 ----- 2) D1 = \$11111111 D2 = \$22222222 SUB.W D1,D2 Result: D2 = \$111111FF D2 = \$22221111
Notes:	

SUBA	Subtract Address . Subtracts the source operand from the destination address register and stores the result in the address register. The size of the operation may be specified as word or long. Word size source operands are sign extended to 32-bit quantities prior to the subtraction.	
Operation	Destination - Source -->; Destination	
Syntax:	SUBA <ea>, An	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected.	
Address Modes:		
Example:	A1= \$00001234 SUBA.W #1111,A1 Mind that with .W the source will get sign extended to 32bit \$00001234 -\$00001111 ----- \$00000123	Result: A1 = \$00000123
Notes:		

SUBI	Subtract Immediate . Subtracts the immediate data from the destination operand, and stores the result in the destination location. The size of the operation may be specified as byte, word, or long. The size of the immediate data matches the operation size.	
Operation	Destination - Immediate Data -->; Destination	
Syntax:	SUBI #<data>, <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Set if the result is zero. Cleared otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a carry is generated. Cleared otherwise.	
Address Modes:		
Example:	D1 = \$11111111 SUBI.W #\$1111,D1 ----- D1 = \$11111111 SUBI.B #\$22,D1	Result: D1 = \$11110000 D1 = \$111111EF
Notes:		

SUBQ	Subtract Quick . Subtracts the immediate data (1-8) from the destination operand, and stores the result in the destination location. The size of the operation may be specified as byte, word, or long.	
Operation	Immediate Data + Destination -->; Destination	
Syntax:	SUBQ #<data>, An;	
Attributes:	Size = (Word, Long)	
Condition Codes:	Not affected	
Address Modes:		

Example:	1) A1= \$00001004 SUBQ.L #8,A1	Result A1 = \$0000FFC:
Notes:	SUBQ is usefull to decrement pointers	

SUBX	Subtract With Extend . Subtracts the source operand from the destination operand and stores the result in the destination location. The operands cas be addressed in two ways: 1. Data register to data register. 2. Memory to memory (Pre-decrement mode) The size of the operation can be specified as byte, word, or long.	
Operation	Destination - Source -->; Destination	
Syntax:	SUBX Dx, Dy SUBX -(Ax), -(Ay)	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Set the same as the carry bit. N Set if the result is negative. Cleared otherwise. Z Cleared if the result is nonzero. Unchanged otherwise. V Set if an overflow is generated. Cleared otherwise. C Set if a carry is generated. Cleared otherwise.	
Address Modes:	Normally the Z condition code bit is set via programming before the start of an operation. This allows successful tests for zero results upon completion of multiple-precision operations.	
Example:		Result:
Notes:		

SWAP	Swap Register Words	
Operation	Register [31:16] \s↔-->; Register [15:0]	
Syntax:	SWAP Dn	
Attributes:	Size = (Word)	
Condition Codes:	X Not affected. N Set if the most significant bit of the 32-bit result is set. Cleared otherwise. Z Set if the 32-bit result is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:	D0= \$12345678 SWAP D0	Result: D0 = \$56781234
Notes:		

TAS	Test Operand And Set . Tests and sets the byte operand addressed by the effective address field. The instruction tests the current value of the operand and sets the N and Z condition bits appropriately. TAS also sets the high-order bit of the operand. The operation uses a locked or read-modify-write transfer sequence. This instruction supports use of a flag or semaphore to coordinate several processors.	
Operation	Destination Tested -->; Condition Codes; 1 -->; bit 7 of Destination	
Syntax:	TAS <ea>	
Attributes:	Size = (Byte)	
Condition Codes:	\c32200 X Not affected. N Set if the most significant bit of the operand is currently set. Cleared otherwise. Z Set if the operand was zero. Cleared otherwise.	

	V Always cleared. C Always cleared.	
Address Modes:		
Example:		Result:
Notes:		

TOUCH		
Operation		
Syntax:		
Attributes:		
Condition Codes:		
Address Modes:		
Example:		Result:
Notes:		

TRAP	Trap. Causes a TRAP #<vector> exception. The instruction adds the immediate operand (vector) of the instruction to 32 to obtain the vector number. The range of vector values is 0-15, which provides 16 vectors.	
Operation	1 -->; Bit of SR *SSP - 2 -->; SSP; Format/Offset -->; (SSP); SSP - 4 -->; SSP; PC -->; (SSP); SSP - 2 -->; SSP; SR -->; (SSP); Vector Address -->; PC	
Syntax:	TRAP #<vector>	
Attributes:	Unsize	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

Trapcc	Trap Conditionally . If the condition is valid, causes a TRAPV exception (vector 7). If the condition is not valid, the processor performs no operation and execution continues with the next instruction.	
Operation	If condition valid then TRAP	
Syntax:	TRAPcc	
Attributes:	Unsize	
Condition Codes:	Not affected.	
Address Modes:		
Example:		Result:
Notes:		

TRAPV	Trap On Overflow . If the overflow condition is set, causes a TRAPV exception (vector 7). If the overflow condition is not set, the processor performs no operation and execution continues with the next instruction.	
Operation	If V then TRAP	

Syntax:	TRAPV
Attributes:	Unsize
Condition Codes:	Not affected.
Address Modes:	
Example:	Result:
Notes:	

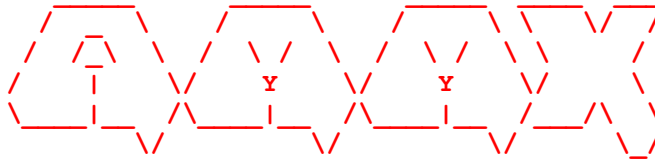
TST	Trap Operand . Compares the operand with zero and sets the condition codes according to the results of the test. The size of the operation is specified as byte, word, or long.	
Operation	Destination Tested -->; Condition Codes	
Syntax:	TST <ea>	
Attributes:	Size = (Byte, Word, Long)	
Condition Codes:	X Not affected. N Set if the operand is negative. Cleared otherwise. Z Set if the operand is zero. Cleared otherwise. V Always cleared. C Always cleared.	
Address Modes:		
Example:	Result:	
Notes:	TST can use ADDRESS REGISTER	

UNLK	Unlink. Loads the stack pointer from the specified address register then loads the address register with the long word pulled from the top of the stack.	
Operation	An -->; SP; (SP) -->; An; SP + 4 -->; SP	
Syntax:	UNLK An	
Attributes:	Unsize	
Condition Codes:	Not affected.	
Address Modes:		
Example:	Result:	
Notes:		

UNPK	Unpack BCD . Places the two BCD digits in the source operand byte into the lower nibbles of two bytes, and places zero bits in the upper nibbles of both bytes. Adds the adjustment value to this unpacked value. Condition codes are not altered. When both operands are data registers, the instruction unpacks the source register contents, adds the extension word, and places the result in the destination register. The high word of the destination register is unaffected. When the specified addressing mode is pre-decrement, the instruction extracts two BCD digits from a byte at the source address. After unpacking the digits and adding the adjustment word, the instruction writes the two bytes to the destination address.	
Operation	Source (Packed BCD) + adjustment -->; Destination (Unpacked BCD)	
Syntax:	UNPK -(Ax), -(Ay), #<adjustment> UNPK Dx, Dy, #<adjustment>	
Attributes:	Unsize	
Condition	Not affected.	

Codes:		
Address Modes:		
Example:		Result:
Notes:		

AMMX



Detecting_AMMX_in_AmigaOS

AMMX is a new feature added to the 68k architecture. Especially the extended registers need to be handled in the multitasking of AmigaOS. As of the Gold 2.7 release, the extensions to context switching are part of the Vampire Support kickstart module. Without this module, AMMX cannot be safely used in a multitasking setting.

Apollo Core presence can be checked in several ways. The recommended method is to check the AttnFlags in ExecBase. The following additions need to be done either in own code or in the respective includes:

```
execbase.i
    IFND AFB_68080
    ;
    ; The AFB_68080 bit is set when a working AC68080
    ; is in the system. If this is set then all bits
    ; for 010/020/030/040 are also set, since the 080
    ; is intended to be compatible with all of them.
    ;
    BITDEF AF,68080,10 ; Set if AC68080
    ENDC
```

```
execbase.h
```

```
#ifndef AFB_68080
/*
 * The AFB_68080 bit is set when a working AC68080
 * is in the system. If this is set then all bits
 * for 010/020/030/040 are also set, since the 080
 * is intended to be compatible with all of them.
 */
#define AFB_68080    10
#define AFF_68080    (1<<10)
#endif
```

ASM-Code to check for Apollo Core may look like the following example:

```
; returns 1 if the program is running on Apollo Core, else 0
    INCLUDE exec/execbase.i
Apollo_Presence:
    move.l    $4.w,a0          ; Load ExecBase
    move.w    AttnFlags(a0),d1 ; Read AttnFlags
    moveq     #1,d0            ;
    btst      #AFB_68080,d1    ; AC68080 ?
    bne.s     _have_apollo
    moveq     #0,d0            ; side note for optimization: Apollo will
                                ; merge a single instruction after a branch
```

```

; to a conditional execution (=free branch)
_have_apollo:
    rts

```

C Code to check for Apollo Core may look like the following example:

```

#include <exec/execbase.h>
#ifndef AFB_68080
#define AFB_68080 10
#endif
#ifndef AFF_68080
#define AFF_68080 (1<<AFB_68080)
#endif
int Apollo_Presence( void )
{
    if( SysBase->AttnFlags & AFF_68080 )
        return 1;

    return 0;
}

```

AMMX1 is guaranteed to be available when AFF_68080 is present in execbase (Apollo Core Gold 2.5 release). There are, however also the instructions introduced with the Gold 2.7 release, in short AMMX2 (PCMP, BSEL, PMIN/PMAX, STOREC, STOREILM, Loadi/Storei, PMULA/PMULL/PMULH etc.). Although manual checks for these instructions would be possible, it is not recommended to do so. The best and shortest way is simply to call `vampire.resource`. The `vampire.resource` has a function that will check for AMMX presence and version and enable it accordingly on request.

SEE ALSO **Enabling_AMMX_in_AmigaOS**

Enabling_AMMX_in_AmigaOS

AMMX is a feature that was added to an 68k processor long after AmigaOS3 development stalled. Hence, the extended registers that come along with Apollo Core are not saved in the regular AmigaOS stackframe. In order to find a balance between backwards compatibility, context switch overhead and Apollo Core support, it was decided that AMMX aware tasks have to announce themselves to the operating system.

Please note at this point: This setting is per task. If your program is using multiple tasks/processes, every one of them needs to call the respective function in `vampire.resource`.

It is not necessary to switch off AMMX explicitly. The functionality to switch off AMMX was provided for library functions to enable/disable AMMX on the fly.

Technical details: AMMX awareness of tasks is signaled in SR, bit 11. Once this flag is set, the task scheduling replacement in the VampireSupport kickstart module will save the extended registers to the user stack. Otherwise, it reverts to the standard stackframe.

Please refer to the calling conventions of `vampire.resource` for further details.

example:

```
#include <vampire/vampire.h>

struct Library *VampireBase;

/* sample program returns 1 if AMMX2 available and activated, 0 if not */
int main( int argc, char ** argv )
{
    if( !(VampireBase = OpenResource( V_VAMPIRENAME ) ) )
        return 0;

    if( VampireBase->lib_Version >= 45 )
    {
        if( V_EnableAMMX( V_AMMX_V2 ) != VRES_ERROR )
            return 1;
    }

    return 0;
}
```

SEE ALSO **vampire/V_EnableAMMX** **vampire/vampire.h** **vampire/vampire.i**

Introduction to AMMX

AMMX, as Gunnar named it, is a 64 Bit SIMD extension to an 68k CPU. Apart from the fact that it shares the 64 Bit width with the MMX of a well-known company, the concept we followed is more geared towards the SIMD extensions in RISC architectures (Altivec, Wireless MMX). In the current state of development, 32 registers are available for SIMD usage. These 32 registers include the well-known D0-D7 (extended to 64 Bit) and 24 new registers which are SIMD exclusive. This way, a lot of work can be done in registers, reducing the strain on memory reads and writes considerably. While reading on, it should get clear that we studied the previously available SIMD architectures but at the same time came up with unique features that separate AMMX from the rest.

Most instructions follow a 3 operand logic D=A op B, where the results of the operation between A and B is stored in any C of the registers. It must be noted at this point that the input operand A doesn't have to be a register. Any effective address in 68k notation is allowed, including immediates.

At this point, some examples are shown:

```
PADDW    D0,D1,D2           ; 4x16 Bit addition D0+D1=D2
PADDW    (A0),D1,D2         ; same, from memory (unaligned)
PADDW    #$8100810081008100,D1,D2 ; add 4x16 Bit constant
PADDW.W  #$8100,D1,D2       ; same as above, with implicit splat
```

The latter two code lines above demonstrate a convenient feature in AMMX. You can specify immediates in AMMX SIMD code, something you don't find easily somewhere else. The constants can be given in full 64 Bit. While this may be useful for some applications, the 64 Bit immediates result in instruction words of 12 Bytes. As an alternative, we added a second way of specifying constants.

The .w Syntax in the last of the example mnemonics triggers the implicit distribution of the immediate data word to all four 16 Bit slots. This way,

the latter two instructions are identical in their arithmetic operation. The difference with implicit splat is a reduction of the instruction word to 6 Bytes.

These two concepts of 3 operand logic and immediates can help to save a number of move instructions that were common to 68k code.

In terms of data movement, two basic operations are supported: LOAD and STORE. While input data for the operations can be gathered by the <VEA> for one of the operands in the arithmetic operations, the destination is a register in the majority of instructions. Therefore, movement to memory needs to be done by STORE.

Example:

```
LOAD    (A0)+,D1;D1=64 bit from any memory location, A0=A0+8
PAVGB   (A1)+,D1,D1 ;8x unsigned byte average (a+b+1)>>1
STORE   D1,(A2)+    ;write result
```

A special case of STORE is also provided, one that can selectively write the individual bytes. The STOREM Rn,Rm,<ea> will only write bytes of which the corresponding mask bit is set (both in MSB to LSB notation).

```
moveq   #4,d3          ;yes yes, this will stall in the following <VEA>
calculation
LOAD    4(A0,D3.l*4),D1 ;D1=64 bit from any memory location
moveq   #%01010101,D2  ;D2.b=bit mask which bytes (bit=1) are to be
written
STOREM  D1,D2,(A2)+    ;write every second byte from D1
```

The third special STORE variant is targeted at 8 Bit pixel data. Typical operations in image/video processing result in intermediate results exceeding the 8 Bit range, which implies clipping before going back to 8 Bit. The Apollo features its own interpretation of PACKUSWB for this purpose. Clipping is done to (0,255). Example:

```
LOAD    (A0)+,D1      ;4 signed words: a0.w a1.w a2.w a3.w
LOAD    (A1)+,D2      ;4 signed words: b0.w b1.w b2.w b3.w
PACKUSWB D1,D2,(A2)+ ;8 unsigned bytes: a0 a1 a2 a3 b0 b1 b2 b3
; operation: vn.b = ( vn.w < 0 ) ? 0 : ( ( vn.w > 255 ) ? 255 : vn.w ); //
n=0...7
```

One catch with SIMD is that you can not always guarantee that you are able to layout your data as needed by the arithmetics. That's why coders have been fond of the permute instruction, introduced with Motorola's PPC7400 (aka G4) series. The Apollo core offers one, too. Two input registers Ra and Rb can be permuted by a given permutation constant into the destination Rd. Example:

```
;byte permutation key semantics for Rm,Rn
; Rm m0 m1 m2 m3 m4 m5 m6 m7 = 0 1 2 3 4 5 6 7
; Rn n0 n1 n2 n3 n4 n5 n6 n7 = 8 9 a b c d e f
;
; ex1: word interleaving
LOAD    (A0)+,D1      ;4 signed words: m0.w m1.w m2.w m3.w
LOAD    (A1)+,D2      ;4 signed words: n0.w n1.w n2.w n3.w
VPERM   #$018923ab,D1,D2,D3 ;D3: m0.w n0.w m1.w n1.w
; ex2: unsigned byte to words
LOAD    (A0),D4       ;8 unsigned bytes m0 m1 m2 m3 m4 m5 m6 m7
```

```

moveq    #0,d5          ;0.1
VPERM    #$F0F1F2F3,D4,D5,D6 ; first four bytes as words m0.w m1.w m2.w m3.w
VPERM    #$F4F5F6F7,D4,D5,D6 ; second four bytes as words m4.w m5.w m6.w m7.w

```

Let's come to arithmetics. Bit-wise operations are:

```

PAND    <VEA>,Rb,Rd
POR     <VEA>,Rb,Rd
PEOR    <VEA>,Rb,Rd
PANDN   <VEA>,Rb,Rd

```

Addition/Subtraction can be done on 8 Bit or 16 Bit.

```

PADDB   <VEA>,Rb,Rd      ;Rd = Rb + <VEA>
PADDW   <VEA>,Rb,Rd      ;
PSUBB   <VEA>,Rb,Rd      ;Rd = Rb - <VEA>
PSUBW   <VEA>,Rb,Rd      ;

```

One special case of add/sub is the BFLYW. A common recurrence in signal transforms (FFT,DCT,DWT) is the butterfly, an operation where the result of an addition and subtraction of two operands is required. In order to augment such transforms, the AMMX offers BFLYW <VEA>,Rb,Rd:Rd+1. Please note that the destination register is actually a consecutive pair (with an even index for the first one).

```

BFLYW D0,D1,D2:D3 ; D2 = D1 + D0 , D3 = D1 - D0 (4 words each)

```

As a side note, we replaced 28 add+sub combinations by butterflies in an 8x8 iDCT, roughly 15% of the total instructions in that function block..

Multiplies are currently offered by the PMUL88 <VEA>,Rb,Rd instruction. It multiplies four words with the given operand and shifts down by 8 Bits after the multiply (Rd = (Rb*<VEA>)>>8). Example:

```

PMUL88.W #16,D0,D1      ; D1 = (D0*16)>>8 = D0/16
PMUL88.W #1024,D0,D1    ; D1 = (D1*1024)>>8 = D0*4
PMUL88.W D2,D3,D4       ;

```

The multiply is implemented with full throughput. The implicit downshift (>>8) can serve as short range shift replacement with the respective multipliers.

Now, a second special operation pair is TRANS. It comes in two flavors, TRANSHi and TRANSLo. Normally, a matrix transpose is quite time consuming as you can only shuffle two operands with other ISA's (just counted 18 instructions for an 8x4 block with 16 bits per element using Intel SSE in an old routine of mine). This normal overhead may well have a significant impact on SIMD execution speed when it comes to matrix operations. Apollo's TRANS operations allow to transpose a 4x4 block with 16 bit per element from row to column order and vice versa.

Example:

```

LOAD (A0)+,E0 ; A0 B0 C0 D0 (A-D = 16 bit words)
LOAD (A0)+,E1 ; A1 B1 C1 D1 (A-D = 16 bit words)
LOAD (A0)+,E2 ; A2 B2 C2 D2 (A-D = 16 bit words)
LOAD (A0)+,E3 ; A3 B3 C3 D3 (A-D = 16 bit words)
;Now transpose the first two words of E0-E3 into the output registers

```

```

TRANSHi E0-E3,E4:E5 ; E4: A0 A1 A2 A3
                        ; E5: B0 B1 B2 B3
;Transpose the lower two words of E0-E3 into the chosen output registers
TRANSLo E0-E3,E6:E7 ; E6: C0 C1 C2 C3
                        ; E7: D0 D1 D2 D3

; Done. Calculate or store...

```

As one can clearly see from the operand list, TRANS is a beast. It takes 32 Bytes as input and provides 16 Bytes output, with a throughput of 1. This requires some compromises. Technically spoken, an Apollo Feature called "late write" is used here. This induces latency. Place instructions for two cycles between a TRANS and the instruction referencing it's result to avoid bubbles. The second compromise concerns input and output registers. The inputs are restricted to a consecutive block of registers, starting with an index dividable by 4, i.e. D0-D3,D4-D7,E0-E3,...,E20-E23. A similar restriction applies to the outputs. Here, the register index must be a multiple of two. TRANS does not accept memory locations or immediates.

Instruction_Words_and_Addressing_Modes

The AMMX instruction words are 32 bit in length. The first word is organized as follows:

```

-----
| Bit      | 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00 |
| Content  | 1  1  1  1  1  1  1  A  B  D <-----VEA-----> |
-----

```

The second word contains the register indices and instruction numbering. Currently, 5 bits are in use for the op itself. The remaining 3 Bits are reserved for future opcodes.

```

-----
| Bit      | 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00 |
| Content  | <- REG-B -> <- REG-D -> 0  0  0 <---- Op ----> |
-----

```

AMMX generally provides the usual m68k addressing modes for one of the operands. In case of operations with clear or typical target into memory, <VEA> applies to the destination. Otherwise, one of the inputs is of the type <VEA>. Exceptions to this scheme are VPERM and TRANS, where memory operands are not allowed.

Before the <VEA> in itself is explained, some other notes towards the Apollo Core. There have been requests to provide additional address registers. As a consequence, Apollo Core offers the additional registers Bn = B0...B7. A number of scalar instructions has been implemented to support the new Bn. In terms of instruction format, these registers were carried over to the AMMX instructions. The distinction between classic addressing modes and the new register set(s) is done by Bits 8,7,6 (=A,B,D) of the instruction word.

The most prominent use of the Bits A,B,D in AMMX is the bank selection for E8-E23. When one of the selector bits is set, E8-E23 are selected instead of D0-D7,E0-E7 for the Operands <REG-B> and <REG-D>. D0-D7 correspond to the bit combinations 0000...0111 and E0-E7 to 1000...1111 in <REG-B/D>. With bank selector bit on, the registers are E8-E23 with the consecutive bit combinations 0000...1111.

The following table lists the valid <VEA> addressing modes for AMMX. Please note that the 68020+ memory indirect modes are not among the valid choices for AMMX. Also, the register direct mode only refers to 64 Bit registers, replacing An/Bn source operands by En.

Another difference between <EA> and <VEA> was chosen in terms of immediates. The default immediate encoding specifies the full 64 bit with four extension words.

With A=1, the short variant with implicit splat is selected.

The other extension words to <VEA> modes are unchanged in comparison to <EA>. Please refer to the 68020+ manual for the common encoding.

+-----+-----+-----+-----+			
MOD REG Effective Addressing Mode in dependency of A-Bit			
+-----+-----+-----+-----+			
		A=0	A=1
+-----+-----+-----+-----+			
000 ---	Dn		E8...E15
001 ---	E0-E7		E16...E23
010 ---	(An)		(Bn)
011 ---	(An)+		(Bn)+
100 ---	-(An)		-(Bn)
101 ---	(d16, An)		(d16, Bn)
+-----+-----+-----+-----+			
110 ---	(d8, An, Xn.SIZE*SCALE)		(d8, Bn, Xn.SIZE*SCALE)
110 ---	(bd, An, Xn.SIZE*SCALE)		(bd, Bn, Xn.SIZE*SCALE)
+-----+-----+-----+-----+			
111 010	(d16, PC)		
111 011	(d8, PC, Xn.SIZE*SCALE)		
111 011	(bd, PC, Xn.SIZE*SCALE)		
111 000	(xxxx).W		
111 001	(xxxxxxxx).L		
111 100	#<xxxxxxxxxxxxxxxx>.q		#<xxxx>.w
+-----+-----+-----+-----+			

AMMX OPCODES

BANK	Register Bank Switching	
Syntax:	BANK SrcA, SrcB, Size	
Example:	BANK 0,0,%10 ; dc.w \$7180 ; exemplary/redundant: ; bank 0 is default lea NUMBERS,a5 ; Load in A5 BANK 0,1,%10 ; dc.w \$7181 lea NUMBERS,a5 ; Load in B5 moveq #0,d0 moveq #0,d1 BANK 0,0,%00 ; dc.w \$7100 ; select register bank 0 ; (redundant, example only) add.l (a5)+,d0 ; BANK 1,0,%00 ; select Address register Bank 1 add.l (a5)+,d1 ; add.l (b5)+,d1 ; D0 should be = D1 = 1	

	<pre> rts numbers: dc.l 1,2 </pre>
Notes:	<pre> ; "Size" is the length of the whole bundle = opcode length + bank_length (2) ; Size = %00 : 4 bytes ; Size = %01 : 6 bytes ; Size = %10 : 8 bytes ; Size = %11 : 10 bytes ; currently valid banks: ; 2 Address register banks (An,Bn) ; 4 Data register banks (D0-D7,E0-E7,E8-E15,E16-E23) BANK MACRO ; ----CCC-DDCCAABB AA BB DD dc.w (%0111000100000000+((\1)*%100)+(\2)+((\3)*%1000000)) ENDM </pre>

MINITERM	Miniterm replicates the Amiga Blitter miniterm calculations
Syntax:	
Example:	
Notes:	Full documentation is in TBD status. Register usage restrictions like TRANSHI/-LO. Inputs are a consecutive set of four registers (D0-D3,D4-D7,E0-E3, etc.). Inputs are assigned as Channels A,B,C,Miniterm. The Output register will carry the result.

LSLQ	64 Bit shift left: LSLQ is a 64 Bit shift left operation. The shift constant in input a is handled as modulo 64. While this operation shares the AMMX encoding, it is a full 64 Bit scalar operation. Zeroes are shifted into the LSBs.
Syntax:	lslq <VEA>,b,d
Example:	
Notes:	<pre> equivalent C code _uint64_t a,b,d; d = b<<a; </pre>

LSRQ	64 Bit shift right: LSRQ is a 64 Bit right left operation. The shift constant in input a is handled as modulo 64. While this operation shares the AMMX encoding style, it is a full 64 Bit scalar operation.
Syntax:	lsrq <VEA>,b,d
Example:	
Notes:	<pre> equivalent C code _uint64_t a,b,d; d = b>>a; </pre>

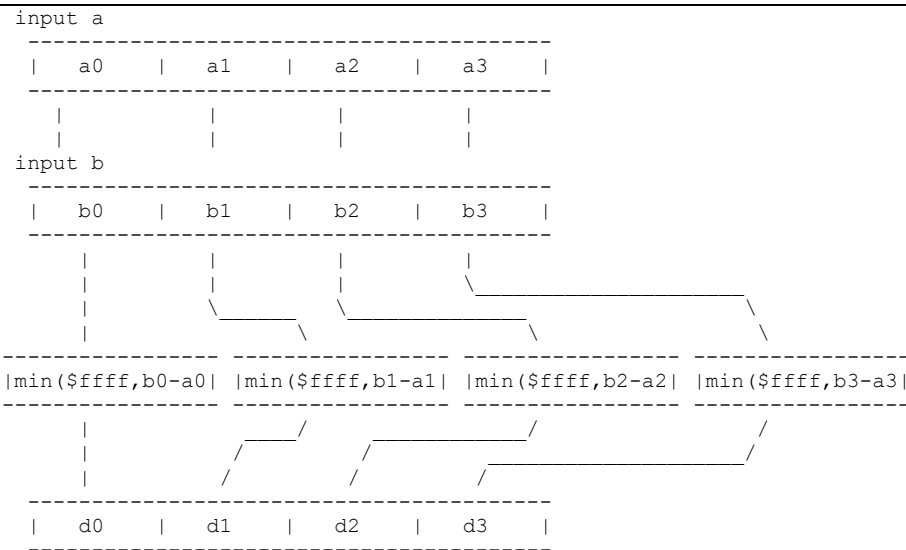
PABS	Each element is converted to absolute value = positive
Operation	PABSB & PABSW
Syntax:	
Attributes:	
Condition Codes:	
Address Modes:	
Example:	
Notes:	

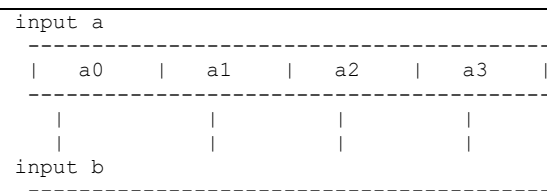
PSUBW	vector subtract short: PADDW is a plain and simple vectorized 16 Bit subtraction operation
--------------	---

	that performs four independent sub.w operations in one shot.
Syntax:	psubw <VEA>,b,d
Example:	
Notes:	input a <pre> ----- a0 a1 a2 a3 ----- </pre> input b <pre> ----- b0 b1 b2 b3 ----- </pre> Diagram illustrating the subtraction operation: Each element of input b is subtracted from the corresponding element of input a to produce the result in d. <pre> ----- b0-a0 b1-a1 b2-a2 b3-a3 ----- </pre> equivalent C Code: <pre> int i; short a[4]; short b[4]; short d[4]; for(i = 0 ; i<4 ; i++) { d[i] = a[i] - b[i]; } </pre>

PADDW	vector add short: PADDW is a plain and simple vectorized 16 Bit addition operation that performs four independent add.w operations in one shot.
Syntax:	paddw <VEA>,b,d
Example:	
Notes:	input a <pre> ----- a0 a1 a2 a3 ----- </pre> input b <pre> ----- b0 b1 b2 b3 ----- </pre> Diagram illustrating the addition operation: Each element of input b is added to the corresponding element of input a to produce the result in d. <pre> ----- b0+a0 b1+a1 b2+a2 b3+a3 ----- </pre> equivalent C Code: <pre> int i; </pre>

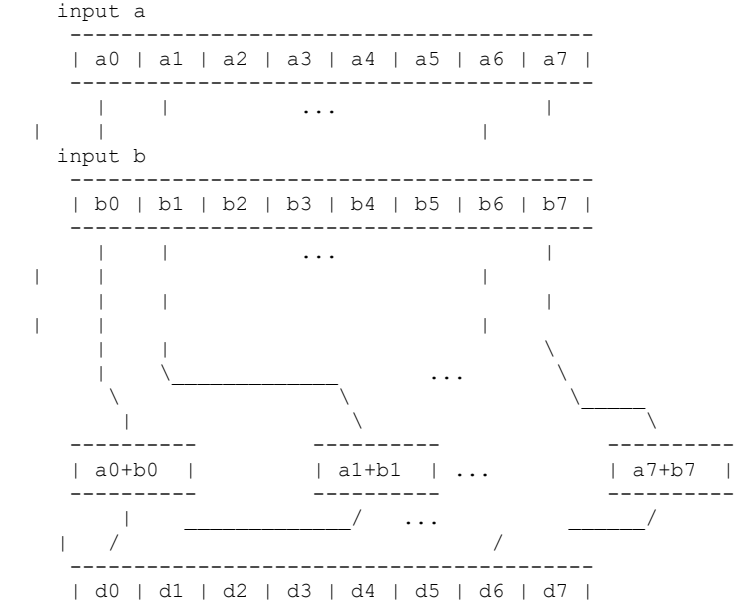
	<pre> short a[4]; short b[4]; short d[4]; for(i = 0 ; i<4 ; i++) { d[i] = a[i] + b[i]; } </pre>
--	---

PSUBUSW	vector subtract unsigned short with saturation: PSUBUSW might come in handy for pixel manipulation with accuracy demands beyond 8 Bit per gun. Subtraction results are implicitly saturated, leaving the complete 16 Bit range available plus the ability to subtract dithering offsets, for example.
Syntax:	psubusw <VEA>,b,d
Example:	
Notes:	 Diagram illustrating the PSUBUSW operation. It shows two 4-element input vectors 'a' and 'b' being subtracted element-wise. The result is then passed through a saturation function $\min(\\$ffff, b[i]-a[i])$ to produce the final output vector 'd'. equivalent C Code: <pre> int i; unsigned short a[4]; unsigned short b[4]; unsigned short d[4]; for(i = 0 ; i<4 ; i++) { d[i] = min(0xffff, b[i] - a[i]); } </pre>

PADDUSW	vector add unsigned short with saturation: PADDUSW might come in handy for pixel manipulation with accuracy demands beyond 8 Bit per gun. Addition results are implicitly saturated, leaving the complete 16 Bit range available plus the ability to add dithering offsets, for example.
Syntax:	paddusw <VEA>,b,d
Example:	
Notes:	 Diagram illustrating the PADDUSW operation. It shows two 4-element input vectors 'a' and 'b' being added element-wise. The result is then passed through a saturation function $\min(\\$ffff, b[i]+a[i])$ to produce the final output vector 'd'.

	equivalent C Code: <pre> int i; unsigned short a[4]; unsigned short b[4]; unsigned short d[4]; for(i = 0 ; i<4 ; i++) { d[i] = min(0xffff, a[i] + b[i]); } </pre>
--	---

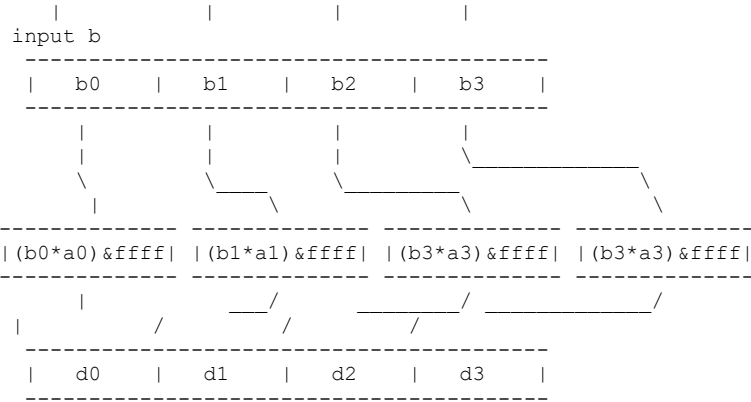
PSUBB	vector subtract bytes: Whenever a byte subtraction is needed and overflows either won't happen or are expected, this is the vector variant of sub.b.
Syntax:	psubb <VEA>,b,d
Example:	
Notes:	equivalent C Code: <pre> int i; char a[8]; /* signed inputs */ char b[8]; char d[8]; for(i = 0 ; i<8 ; i++) { d[i] = b[i] - a[i]; } </pre>

PADDB	vector add bytes: Whenever a byte addition is needed and overflows either won't happen or are expected, this is the vector variant of add.b.
Syntax:	paddb <VEA>,b,d
Example:	
Notes:	 <pre> equivalent C Code: int i; char a[8]; /* signed inputs */ char b[8]; char d[8]; for(i = 0 ; i<8 ; i++) { d[i] = a[i] + b[i]; } </pre>

PSUBUSB	vector subtract unsigned bytes with unsigned saturation: Psubusb is useful when dealing with 8 Bit pixel data. All subtractions are guaranteed to be kept in the 8 Bit range. Subtraction results <0 are saturated to 0.
Syntax:	psubusb <VEA>,b,d
Example:	
Notes:	 <pre> equivalent C Code: int i; char a[8]; /* unsigned inputs */ char b[8]; char d[8]; for(i = 0 ; i<8 ; i++) { d[i] = (a[i] > b[i]) ? a[i] - b[i] : 0; } </pre>

	task. Just preload 255-Alpha in the relevant byte slots of b, destination content in a and RGB (pre-multiplied with Alpha) into d.
Syntax:	pmula <VEA>,b,d
Example:	
Notes:	<pre> input a ----- a0 a1 a2 a3 a4 a5 a6 a7 ----- ----- input b ----- b0 b1 b2 b3 b4 b5 b6 b7 ----- ----- input/output d ----- d0 d1 d2 d3 d4 d5 d6 d7 ----- ----- ((b0*a0)>>8)+d0 ((b1*a1)>>8)+d1 ((b7*a7)>>8)+d7 ----- ----- d0 d1 d2 d3 d4 d5 d6 d7 ----- </pre> equivalent C Code: <pre> int i; unsigned char a[8]; unsigned char b[8]; unsigned char d[8]; for(i = 0 ; i<8 ; i++) { d[i] = ((a[i] * b[i]) >> 8) + d[i]; } </pre>

PMULL	vector multiply short and keep lower 16 Bit: PMULL mostly applies to cases where the result is known to fit in 16 Bit. Usually, it is expected that PMULL is used in tandem with PMULH to obtain a 16x16 = 32 Bit result. Another application for PMULL is to serve as a left shift operator. Being a multiplication instruction, pass 2^(shift value) instead of the number of bits.
Syntax:	pmull <VEA>,b,d
Example:	<pre> pmull.w #54,E0,E2 ; E2 = E0*54 pmull.w #1024,E0,E3 ; E3 = E0<<10 ;16*16 = 32 Bit multiplication example pmull.w E4,E5,E6 ;low part l0 l1 l2 l3 pmulh.w E4,E5,E7 ;high part h0 h1 h2 h3 vperm #018923ab,E7,E6,E8 ; h0 l0 h1 l1 vperm #045cd67ef,E7,E6,E9 ; h2 l2 h3 l3 </pre>
Notes:	<pre> input a ----- a0 a1 a2 a3 ----- ----- </pre>



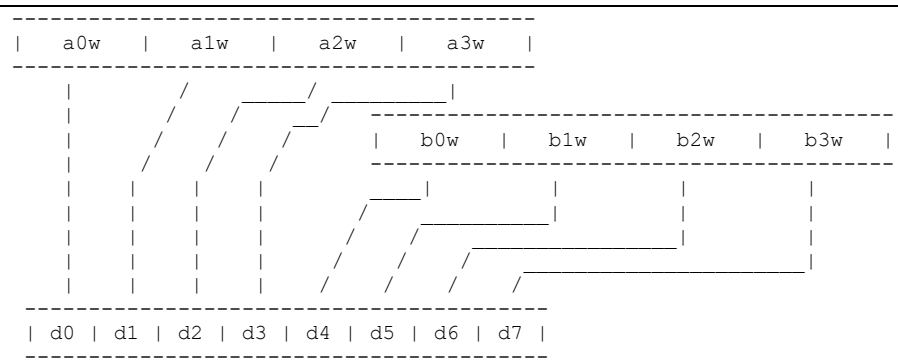
equivalent C Code:

[illegible]

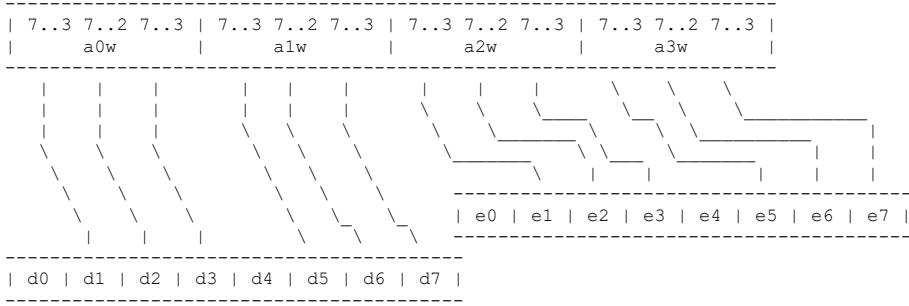
PMULH	vector multiply short and keep upper 16 Bit: PMULH is useful on it's own in cases where one of the operands is a fixed-point fractional variable with an absolute value <1. In said cases, the usual renormalizing shift is not necessary. Application cases like trigonometric transforms and 3D matrix transforms come to mind. PMULH teams up with PMULL when $16 \times 16 = 32$ Bit results are of interest.
Syntax:	pmulh <VEA>,b,d
Example:	
Notes:	Diagram illustrating the PMULH operation: Input a: a_0, a_1, a_2, a_3 (each 16 bits) Input b: b_0, b_1, b_2, b_3 (each 16 bits) Operation: $(b_0 * a_0) \gg 16, (b_1 * a_1) \gg 16, (b_2 * a_2) \gg 16, (b_3 * a_3) \gg 16$ Output: d_0, d_1, d_2, d_3 (each 16 bits) equivalent C Code: <pre>int i; short a[4]; /* signed inputs */ short b[4]; short d[4];</pre>

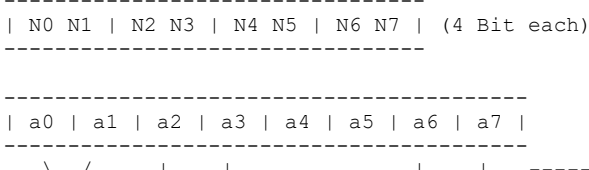
	<pre> for(i = 0 ; i<4 ; i++) { d[i] = (a[i] * b[i]) >> 16; } </pre>
--	--

PMUL88	vector multiply short and shift down 8: PMUL88 is effectively a multiply of a 16.0 integer with an 8.8 fixed point. As such, it can be used for short-range shifts as well. Should accuracy and dynamic range not fit the particular needs, please have a look at PMULL and PMULH.
Syntax:	pmul88 <VEA>,b,d
Example:	pmul88.w #64,E0,E1 ; E1 = (E0*64)>>8 = E0>>2 pmul88.w #1024,E0,E2 ; E2 = (E0*1024)>>8 = E0<<2
Notes:	 <pre> equivalent C Code: int i; short a[4]; /* signed inputs */ short b[4]; short d[4]; for(i = 0 ; i<4 ; i++) { d[i] = (a[i] * b[i]) >> 8; } </pre>

PACKUSWB	Pack 2x4 signed shorts into 8 unsigned char, saturate to 0..255: filter calculations in 16 bit, output of 8 Bit unsigned pixel data as a result
Syntax:	packuswb a,b,<VEA>
Example:	
Notes:	

	equivalent C Code: <pre> int i; short a[4]; short b[4]; unsigned char d[8]; for(i = 0 ; i<4 ; i++) { d[i] = (a[i] < 0) ? 0 : (a[i] > 255) ? 255 : a[i]; d[i+4] = (b[i] < 0) ? 0 : (b[i] > 255) ? 255 : b[i]; } </pre> The destination can be either a register or memory location. Immediate operands don't apply here.
--	---

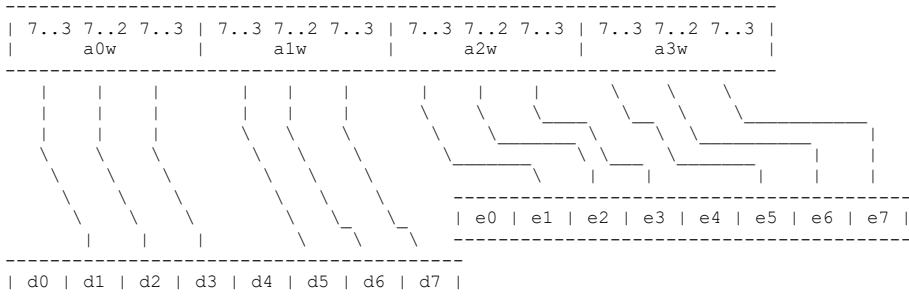
PACK3216	pack 32 Bit ARGB data into 16 Bit RGB565: Conversion of ARGB input data to HiColor screen and texture formats.
Syntax:	pack3216 a,b,<VEA>
Example:	
Notes:	 constraints: The destination register pair needs to be consecutive, starting with an even register index (e.g. unpack1632 (a0),E0:E1). Immediate operands don't apply here. equivalent C Code: <pre> int i; unsigned short a[4]; unsigned char d[16]; for(i = 0 ; i<4 ; i++) { d[i*4] = 0xff; d[i*4+1] = ((a[i]>>8) & 0xf8) ((a[i]>>13) & 0x7); d[i*4+2] = ((a[i]>>3) & 0xfc) ((a[i]>>9) & 0x3); d[i*4+3] = ((a[i]<<3) & 0xf8) ((a[i]>>2) & 0x7); } </pre>

VPERM	permute the contents of two registers into destination register: Flexible shuffling of bytes can be necessary virtually everywhere in SIMD-World.
Syntax:	vperm #N,a,b,d
Example:	; convert 8 Bit pixel data to 16 Bit load (a0),E1 ;load 8 Bytes moveq #0,d0 ;D0 = 0 (lower 32 Bit) vperm #\$48494a4b,d0,E1,E2 ; first 4 Bytes of E1 into 4 Words (insert zeros from d0) vperm #\$4c4d4e4f,d0,E1,E3 ; second 4 Bytes of E1 into 4 Words
Notes:	

	<pre> ----- a2 a3 b2 b3 c2 c3 d2 d3 ----- </pre> constraints: This instruction does not support memory operands. The four inputs (a-d) and the destination (e,f) must be consecutive registers. The first source a is constrained to a multiple of 4 (i.e. D0-D3,D4-D7,E0-E3,...,E20-E23). The destination register index pair (e,f) are restricted to a multiple of two (i.e. D0:D1,D2:D3 etc.). equivalent C Code: <pre> int i,n; unsigned short a[16]; /* 4 registers of 8 bytes each */ unsigned short e[4]; /* 1 register of 8 bytes */ unsigned short f[4]; /* 1 register of 8 bytes */ for(i = 0 ; i<4 ; i++) { e[i] = a[4*i]; f[i] = a[4*i + 1]; } </pre>
--	--

TRANSLO	matrix transposition, lower half: This instruction allows a 4x4 matrix transposition in just two CPU cycles (in conjunction with transhi). The extension towards larger matrices (8x8,16x16 etc.) is straightforward.
Syntax:	translo a-d , e:f
Example:	load (a0),E0 load 8(a0),E1 load 16(a0),E2 load 24(a0),E3 transhi E0-E3,E4:E5 ;transpose upper (left) half of matrix (to complete example) translo E0-E3,E6:E7 ;transpose lower (right) half of matrix
Notes:	input: 4 consecutive registers <pre> ----- a0 a1 a2 a3 a4 a5 a6 a7 ----- ----- b0 b1 b2 b3 b4 b5 b6 b7 ----- ----- c0 c1 c2 c3 c4 c5 c6 c7 ----- ----- d0 d1 d2 d3 d4 d5 d6 d7 ----- </pre> output: two consecutive registers <pre> ----- a4 a5 b4 b5 c4 c5 d4 d5 ----- ----- a6 a7 b6 b7 c6 c7 d6 d7 ----- </pre> constraints: This instruction does not support memory operands. The four inputs (a-d) and the destination (e,f) must be consecutive registers. The first source a is constrained to a multiple of 4 (i.e. D0-D3,D4-D7,E0-E3,...,E20-E23). The destination register index pair (e,f) are restricted to a multiple of two (i.e. D0:D1,D2:D3 etc.).

	<pre> equivalent C Code: int i; unsigned short a[16]; /* 4 registers of 8 bytes each */ unsigned short e[4]; /* 1 register of 8 bytes */ unsigned short f[4]; /* 1 register of 8 bytes */ for(i = 0 ; i<4 ; i++) { e[i] = a[4*i + 2]; f[i] = a[4*i + 3]; } </pre>
--	---

UNPACK1632	unpack 16 Bit RGB565 data into 32 Bit ARGB Conversion of RGB565 HiColor input data to ARGB screen and texture formats.
Syntax:	unpack1632 <VEA>,d:d+1
Example:	
Notes:	 constraints: The destination register pair needs to be consecutive, starting with an even register index (e.g. unpack1632 (a0),E0:E1). Immediate operands don't apply here. equivalent C Code: <pre> int i; unsigned short a[4]; unsigned char d[16]; for(i = 0 ; i<4 ; i++) { d[i*4] = 0xff; d[i*4+1] = ((a[i]>>8) & 0xf8) ((a[i]>>13) & 0x7); d[i*4+2] = ((a[i]>>3) & 0xfc) ((a[i]>>9) & 0x3); d[i*4+3] = ((a[i]<<3) & 0xf8) ((a[i]>>2) & 0x7); } </pre>

BFLYW	butterfly operation, single-cycle vector short addition _and_ subtraction Signal transforms like FFT, DCT, DWT etc. and lifting approaches in terms of filter banks in general.
Syntax:	bflyw <VEA>,b,d:d+1
Example:	
Notes:	inputs: <pre> ----- a0w a1w a2w a3w ----- ----- b0w b1w b2w b3w ----- outputs: ----- b0w+a0w b1w+a1w b2w+a2w b3w+a3w ----- ----- </pre>

	<pre> b0w-a0w b1w-a1w b2w-a2w b3w-a3w ----- constraints: The destination register pair needs to be consecutive, starting with an even register index (e.g. bflyw (a0),E8,E0:E1). Immediate operands don't apply here. The destination can be either a register or memory location. Immediate operands don't apply here. equivalent C Code: int i; short a[4]; /* */ short b[4]; /* */ short d[4]; /* */ short e[4]; /* */ for(i = 0 ; i<4 ; i++) { d[i] = b[i] + a[i]; f[i] = b[i] - a[i]; } </pre>
--	--

C2P	chunky to planar conversion, bit-wise transpose Chunky-to-planar conversion and vice versa. Consider the combination of C2P and VPERM/TRANSHI/TRANSLO for efficient implementations. Also useful to extract condition masks from AMMX registers (e.g. after PCMP) into regular data registers.
Syntax:	c2p <VEA>,d
Example:	
Notes:	<pre> input (bytes k-r with their bits 7...0): ----- kkkkkkkk llllllll mmmmmmmm nnnnnnnn ooooooooo pppppppp qqqqqqqq rrrrrrrr 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- output: ----- klmnopqr klmnopqr klmnopqr klmnopqr klmnopqr klmnopqr klmnopqr klmnopqr 77777777 66666666 55555555 44444444 33333333 22222222 11111111 00000000 ----- equivalent C Code: int i,j; unsigned char t; unsigned char a[8]; unsigned char d[8]; for(i = 0 ; i<8 ; i++) { for(j=0,t=0 ; j < 8 ; j++) t = ((a[j] >> (7-i)) & 1) << (7-j); d[i] = t; } </pre>

BSEL	This instruction allows a bit-by-bit selection of data from two sources into the destination. Typically, this is applied in conjunction with a prior pcmp instruction. Tasks like conditional replenishment and clipping easily come to mind.
Syntax:	bsel <VEA>,b,d
Example:	
Notes:	<pre> input1: <VEA> (bytes l-s with their bits 7...0) ----- llllllll mmmmmmmm nnnnnnnn ooooooooo pppppppp qqqqqqqq rrrrrrrr ssssssss 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- input3: (contents of destination register d, with bytes d-k) ----- dddddd eeeeeeee ffffffff gggggggg hhhhhhhh iiiiiiiii jjjjjjjj kkkkkkkk </pre>

	<pre> 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- input2: (contents of register b with bytes b,c,t-y) ----- bbbbbbbb cccccccc tttttttt uuuuuuuu vvvvvvvv wwwwwww xxxxxxxx yyyyyyyy 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- ----- (b7&17) (!b7&d7) .. (b0&10) (!b0&d0) .. (y7&s7) (!y7&k7) .. (y0&s0) (!y0&k0) ----- ----- dddddddd eeeeeeee ffffffff gggggggg hhhhhhh iiiiiii jjjjjjj kkkkkkkk 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- equivalent C Code: { unsigned long long a; unsigned long long b; unsigned long lond d; d = (d & (!b)) (a & b); /* BSEL a,b,d */ } </pre>
--	---

PAND_POR_PE OR_PANDN	64 Bit logic operations EOR,AND,OR AND I think NOT that I have to write more. :-)
Syntax:	pand <VEA>,b,d por <VEA>,b,d peor <VEA>,b,d pandn <VEA>,b,d
Example:	
Notes:	<pre> input1: <VEA> (bytes l-s with their bits 7...0) ----- 11111111 mmmmmmmm nnnnnnnn oooooooo pppppppp qqqqqqq rrrrrrrr ssssssss 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- input2: (contents of register b with bytes b,c,t-y) ----- bbbbbbbb cccccccc tttttttt uuuuuuuu vvvvvvvv wwwwwww xxxxxxxx yyyyyyyy 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- ----- b7&17 .. b0&10 (partial PAND ILLUSTRATION) s7&y7 .. s0&y0 ----- ----- dddddddd eeeeeeee ffffffff gggggggg hhhhhhh iiiiiii jjjjjjj kkkkkkkk 76543210 76543210 76543210 76543210 76543210 76543210 76543210 76543210 ----- equivalent C Code: { unsigned long long a; unsigned long long b; unsigned long lond d; </pre>

	<pre> d = a & b; /* PAND */ d = a b; /* POR */ d = a ^ b; /* PEOR */ d = !a & b; /* PANDN */ </pre>
--	---

PAVGB	average 8 unsigned bytes with 8 unsigned bytes Simple half-sample linear interpolation. This type of operation is used (read: mandatory) in many classic video codecs, ranging from MPEG-1 to MPEG-4 part 2.
Syntax:	pavgb <VEA>,b,d
Example:	
Notes:	 <pre> equivalent C Code: int i; unsigned char a[8]; unsigned char b[8]; unsigned char d[8]; for(i = 0 ; i<8 ; i++) { d[i] = (a[i] + b[i] + 1) >> 1; } </pre>

LOAD	load 64 bit into destination register Load is the AMMX equivalent to move <ea>,dn. As most other AMMX instructions, it can load either from memory (any An,Bn addressing modes), from another register or constants. In case of memory sources, there are no restrictions towards alignment. You may load from any valid address in Chip- and FastRAM.
Syntax:	load <VEA>,d
Example:	<pre> load (a0),E0 load 1(a0),E1 load #\$c0ffee00feedface,E2 load.w #\$beef,E3 ; this one is with implicit splat ; E3 = \$beefbeefbeefbeef </pre>
Notes:	<pre> equivalent C Code: { unsigned long long src; unsigned long long d; d = src; } </pre>

LOADi	load 64 bit indirect into destination register For many cases, the normal load instruction is more appropriate and convenient. While this indexed variant requires to preload the index register, it helps for example at places where the contents of the destination register are to be changed conditionally. Also, you may think of preloading AMMX registers in a loop instead of in a row to keep code size small (where appropriate).
Syntax:	loadi <VEA>,d
Example:	moveq #1,d0

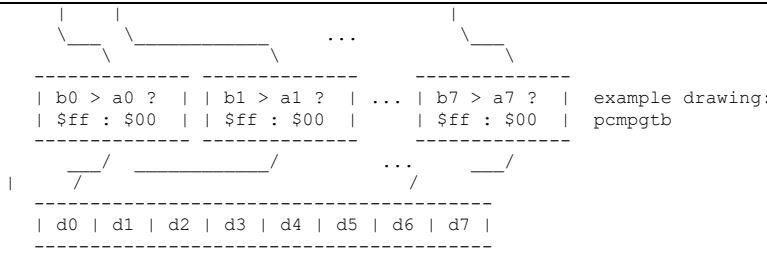
	<pre>loadi (a0),d0;register number 1 is "d1", so d1=(a0) moveq #7,d0 ;example: preload E0-E7 moveq #40,d1 ;= E0 index .loop: loadi (a0)+,d1 addq.l #1,d1 dbf d0,.loop register map (partial, decimal numbers): 00 - 07 = D0 - D7 08 - 15 = A0 - A7 16 - 23 = B0 - B7 40 - 47 = E0 - E7 48 - 55 = E8 - E15 56 - 63 = E16 - E23</pre>
Notes:	<pre>equivalent C Code: { unsigned long long src; unsigned char d; unsigned long long regfile[64]; /* Apollo-internal register file */ regfile[d] = src; } notes: In reality, the mnemonic should be "loadi <VEA>, (d)". The interim implementation in VASM does not reflect the indirect reference to "d", however. So in the meantime, think of "(d)" while specifying "d" in code.</pre>

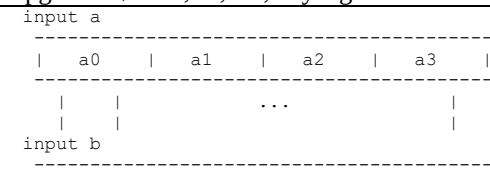
STORE	store 64 bit from source register in memory Store is the AMMX equivalent to move dn,<ea>. It unconditionally writes 64 Bit into the destination. In typical cases, the destination is some location in memory. You may store to any valid address in Chip- and FastRAM. There are no alignment restrictions.
Syntax:	store a,<VEA>
Example:	<pre>store E0,(a0) store E1,9(a0)</pre>
Notes:	<pre>{ unsigned long long a; unsigned long long dest; dest = a; }</pre>

STOREi	store 64 bit indirect source into memory location For many cases, the normal store instruction is more appropriate and convenient. While this indexed variant requires to preload the index register, it helps for example at places where the source register is to be changed conditionally. Also, you may think of storing a list of AMMX registers in a loop instead of in a row to keep code size small (where appropriate).
Syntax:	storei a,<VEA>
Example:	<pre>moveq #1,d0 storei d0,(a0) ;register number 1 is "d1", so (a0) = d1 register map (partial, decimal numbers): 00 - 07 = D0 - D7 08 - 15 = A0 - A7 16 - 23 = B0 - B7 40 - 47 = E0 - E7</pre>

	48 - 55 = E8 - E15 56 - 63 = E16 - E23
Notes:	equivalent C Code: <pre> { unsigned long long dest; unsigned char a; unsigned long long regfile[64]; /* Apollo-internal register file */ dest = regfile[d]; } </pre> notes: In reality, the mnemonic should be "storei (d),<VEA>". The interim implementation in VASM does not reflect the indirect reference to "d", however. So in the meantime, think of "(d)" while specifying "d" in code.

STOREm	store a selection of bytes from a into destination It is usual in SIMD that you can write only the native amount of bytes at once. This instruction enables selective overwriting of memory, based on the contents of the mask register (lower 8 Bit). This instruction has several uses. Perhaps the most prominent application is for cookie-cut: selective writing of pixels from sprites to the screen.
Syntax:	storem a,m,<VEA>
Example:	<pre> move.b #\$ff,d0 storem E0,d0,(a0) ;this example is the same as store E0,(a0) move.b #\$f0,d0 storem E0,d0,8(a0) ;overwrite 4 bytes from 8(a0) with upper 32 bit of E0 </pre> ; one way of color key (see also storeilm) <pre> pcmpeqw.w #\$f81f,E0,E2 ;magenta HiColor RGB565 pixel(s) in E0 ? c2p e2,e2 ;re-order: get bits from word mask into one byte peor.w #\$ffff,e2,e2 ;negate mask (logical: !magenta) storem E0,E2,(a0) ;store only words where the magenta check didn't match </pre>
Notes:	input a <pre> ----- a0 a1 a2 a3 a4 a5 a6 a7 ----- ----- logical input: destination contents ----- d0 d1 d2 d3 d4 d5 d6 d7 ----- ----- input m: lower 8 bit of second argument ----- m ----- ----- (m&128) ? (m&64) ? ... (m&1) ? a0 : d0 a1 : d1 a7 : d7 ----- d0 d1 d2 d3 d4 d5 d6 d7 ----- </pre> equivalent C Code: <pre> int i; unsigned char a[8]; unsigned char m; unsigned char d[8]; for(i = 0 ; i<8 ; i++) { d[i] = (m & (1<<(7-i))) ? a[i] : d[i]; } </pre>

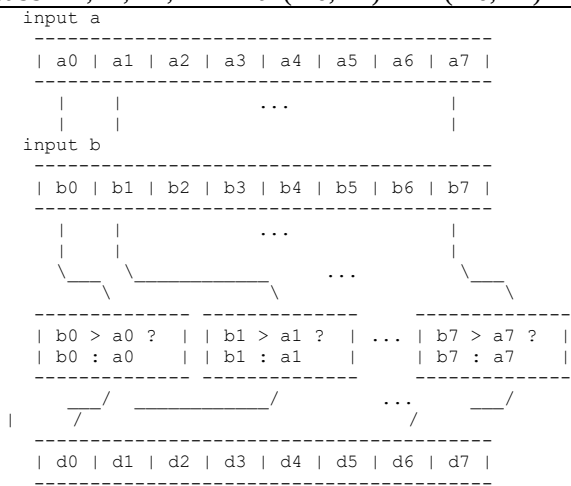
	 equivalent C Code: <pre> int i; unsigned char au[8]; /* unsigned inputs */ unsigned char bu[8]; char as[8]; /* signed inputs */ char bs[8]; unsigned char d[8]; /* all 4 variants shown in this loop, in practice only one condition apply per instruction */ for(i = 0 ; i<8 ; i++) { /* pcmptgb, applies to au/bu and as/bs */ d[i] = (bu[i] == au[i]) ? 0xff : 0x00; d[i] = (bs[i] == as[i]) ? 0xff : 0x00; /* pcmptgb */ d[i] = (bs[i] > as[i]) ? 0xff : 0x00; /* pcmptgb */ d[i] = (bs[i] >= as[i]) ? 0xff : 0x00; /* pcmphib */ d[i] = (bu[i] > au[i]) ? 0xff : 0x00; } </pre>
--	--

PCMPccW	short-by-short vector compare Comparisons are as important in SIMD as in scalar code. The difference to the latter is that condition codes are not set in SIMD. Instead, you get bit masks. In the easiest case, you can just BSEL for handling of true/false conditions. Also, STOREILM might be worth a look. In case that scalar handling of vector condition masks is desired, C2P with a Dn register target might come in handy.
Syntax:	pcmptqw <VEA>,b,d pcmptgw <VEA>,b,d pcmptgew <VEA>,b,d pcmpthiw <VEA>,b,d
Example:	pcmptqw.w #0101,E0,E1 ; any word slot in E0 equal to a constant of "1" ? ; ;pcmpths calculation pcmptqw E3,E0,E1 ; E1: E0 == E3 ? pcmpthiw E3,E0,E2 ; E2: E0 > E3 ? (unsigned) por E1,E2,E1 ; E1: E0 >= E3 ? (unsigned) ;pcmpths when E3 is always >= 1 psubsw.w #0101,E3,E2 pcmpthiw E2,E0,E1 ; E1: E0 >= E3 ? ; pcmptgw #0404,E0,E1 ; any signed short slot in E0 > \$40 ? pcmptgew #0404,E0,E1 ; any signed short slot in E0 >= \$40 ?
Notes:	

	example drawing: pcmpeqw equivalent C Code: <pre> int i; unsigned short au[4]; /* unsigned inputs */ unsigned short bu[4]; short as[4]; /* signed inputs */ short bs[4]; unsigned short d[4]; /* all 4 variants shown in this loop, in practice only one condition apply per instruction */ for(i = 0 ; i<4 ; i++) { /* pcmpeqw, applies to au/bu and as/bs */ d[i] = (bu[i] == au[i]) ? 0xffff : 0x0000; d[i] = (bs[i] == as[i]) ? 0xffff : 0x0000; /* pcmpeqw */ d[i] = (bs[i] > as[i]) ? 0xffff : 0x0000; /* pcmpeqw */ d[i] = (bs[i] >= as[i]) ? 0xffff : 0x0000; /* pcmphiw */ d[i] = (bu[i] > au[i]) ? 0xffff : 0x0000; } </pre>
--	--

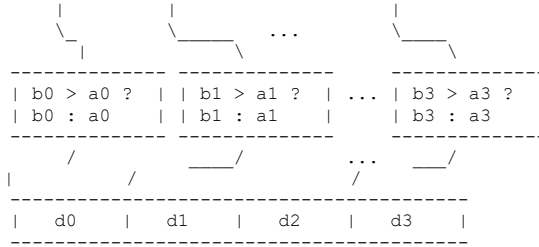
PMINxB	byte-by-byte vector compare and obtain smaller PMIN allows to simplify clipping operations. Normally, one would write a pcmp/bsel sequence. This construct by PMAX reduces the operation to one instruction only.
Syntax:	pminub <VEA>,b,d pminsb <VEA>,b,d
Example:	pmaxub E0,E1,E2 ; max(E0, E1) pminub E0,E1,E1 ; min(E0, E1) psubb E1,E2,E1 ; E1 = max(E0,E1) - min(E0,E1) = abs(E0-E1)
Notes:	

	<pre> equivalent C Code: int i; unsigned char au[8]; /* unsigned inputs */ unsigned char bu[8]; char as[8]; /* signed inputs */ char bs[8]; unsigned char d[8]; /* both variants shown in this loop, in practice only one condition apply per instruction */ for(i = 0 ; i<8 ; i++) { /* pminsb */ d[i] = (bs[i] < as[i]) ? bs[i] : as[i]; /* pminub */ d[i] = (bu[i] < au[i]) ? bu[i] : au[i]; } </pre>
--	---

PMAXxB	byte-by-byte vector compare and obtain larger PMAX allows to simplify clipping operations. Normally, one would write a pcmp/bsel sequence. This construct by PMAX reduces the operation to one instruction only.
Syntax:	<pre> pmaxub <VEA>,b,d pmaxsb <VEA>,b,d </pre>
Example:	<pre> pmaxub E0,E1,E2 ; max(E0, E1) pminub E0,E1,E1 ; min(E0, E1) psubb E1,E2,E1 ; E1 = max(E0,E1) - min(E0,E1) = abs(E0-E1) </pre>
Notes:	 <pre> equivalent C Code: int i; unsigned char au[8]; /* unsigned inputs */ unsigned char bu[8]; char as[8]; /* signed inputs */ char bs[8]; unsigned char d[8]; /* both variants shown in this loop, in practice only one condition apply per instruction */ for(i = 0 ; i<8 ; i++) { /* pmaxsb */ d[i] = (bs[i] > as[i]) ? bs[i] : as[i]; /* pmaxub */ d[i] = (bu[i] > au[i]) ? bu[i] : au[i]; } </pre>

PMINxW	short-by-short vector compare and obtain smaller PMIN allows to simplify clipping operations. Normally, one would write a pcmp/bse sequence. This construct by PMAX reduces the operation to one instruction only.
Syntax:	pminuw <VEA>,b,d pminsw <VEA>,b,d
Example:	pmaxuw E0,E1,E2 ; max(E0, E1) pminuw E0,E1,E1 ; min(E0, E1) psubw E1,E2,E1 ; E1 = max(E0,E1) - min(E0,E1) = abs(E0-E1)
Notes:	input a <pre> ----- a0 a1 a2 a3 ----- ----- </pre> input b <pre> ----- b0 b1 b2 b3 ----- ----- </pre> Diagram illustrating the PMIN operation. It shows two input vectors, 'input a' and 'input b', each with four elements (a0, a1, a2, a3 and b0, b1, b2, b3). The operation compares corresponding elements (a0 < b0, a1 < b1, etc.) and selects the smaller value for the output vector 'd' (d0, d1, d2, d3). The diagram shows the comparison results (b0 < a0 ?, b1 < a1 ?, etc.) and the resulting output values (d0, d1, d2, d3). equivalent C Code: <pre> int i; unsigned short au[4]; /* unsigned inputs */ unsigned short bu[4]; short as[4]; /* signed inputs */ short bs[4]; short d[4]; /* both variants shown in this loop, in practice only one condition apply per instruction */ for(i = 0 ; i<4 ; i++) { /* pminsw */ d[i] = (bs[i] < as[i]) ? bs[i] : as[i]; /* pminuw */ d[i] = (bu[i] < au[i]) ? bu[i] : au[i]; } </pre>

PMAXxW	short-by-short vector compare and obtain larger PMAX allows to simplify clipping operations. Normally, one would write a pcmp/bse sequence. This construct by PMAX reduces the operation to one instruction only.
Syntax:	pmaxuw <VEA>,b,d pmaxsw <VEA>,b,d
Example:	pmaxuw E0,E1,E2 ; max(E0, E1) pminuw E0,E1,E1 ; min(E0, E1) psubw E1,E2,E1 ; E1 = max(E0,E1) - min(E0,E1) = abs(E0-E1)
Notes:	input a <pre> ----- a0 a1 a2 a3 ----- ----- </pre> input b <pre> ----- b0 b1 b2 b3 ----- ----- </pre>



equivalent C Code:

```
int i;
unsigned short au[4]; /* unsigned inputs */
unsigned short bu[4];
short as[4]; /* signed inputs */
short bs[4];
short d[4];

/* both variants shown in this loop, in practice only one condition
   apply per instruction */
for( i = 0 ; i<4 ; i++ )
{
    /* pmaxsw */
    d[i] = ( bs[i] > as[i] ) ? bs[i] : as[i];

    /* pmaxuw */
    d[i] = ( bu[i] > au[i] ) ? bu[i] : au[i];
}
```

SAGA Information

Address	Name	Access	OCS	ECS	AGA	SAGA	Description
DFF000	BLTDDAT	ER	*	*	*	*	Blitter dest. early read (dummy address)
DFF002	DMACONR	R	*	*	*	*	Dma control (and blitter status) read
DFF004	VPOSR	R	*	*	*	*	Read vertical most sig. bits (and frame flop)
DFF006	VHPOSR	R	*	*	*	*	Read vert and horiz position of beam
DFF008	DSKDATR	ER	*	*	*	*	Disk data early read (dummy address)
DFF00A	JOYODAT	R	*	*	*	*	Joystick-mouse 0 data (vert, horiz)
DFF00C	JOYIDAT	R	*	*	*	*	Joystick-mouse 1 data (vert, horiz)
DFF00E	CLXDAT	R	*	*	*	*	Collision data reg. (read and clear)
DFF010	ADKCONR	R	*	*	*	*	Audio,disk control register read
DFF012	POTODAT	R	*	*	*	*	Pot counter data left pair (vert, horiz)
DFF014	POTIDAT	R	*	*	*	*	Pot counter data right pair (vert, horiz)
DFF016	POTINP	R	*	*	*	*	Pot pin data read
DFF018	SERDATR	R	*	*	*	*	Serial port data and status read
DFF01A	DSKBYTR	R	*	*	*	*	Disk data byte and status read
DFF01C	INTENAR	R	*	*	*	*	Interrupt enable bits read
DFF01E	INTREQR	R	*	*	*	*	Interrupt request bits read
DFF020	DSKPTH	W	*	*	*	*	Disk pointer (high 5 bits, was 3 bits)
DFF022	DSKPTL	W	*	*	*	*	Disk pointer (low 15 bits)
DFF024	DSKLEN	W	*	*	*	*	Disk length
DFF026	DSKDAT	W	*	*	*	*	Disk DMA data write
DFF028	REFPTR	W	*	*	*	*	Refresh pointer
DFF02A	VPOSW	W	*	*	*	*	Write vert most sig. bits (and frame flop)
DFF02C	VHPOSW	W	*	*	*	*	Write vert and horiz pos of beam
DFF02E	COPCON	W	*	*	*	*	Coprocessor control
DFF030	SERDAT	W	*	*	*	*	Serial port data and stop bits write
DFF032	SERPER	W	*	*	*	*	Serial port period and control
DFF034	POTGO	W	*	*	*	*	Pot count start,pot pin drive enable data
DFF036	JOYTEST	W	*	*	*	*	Write to all 4 joystick-mouse counters at once
DFF038	STREQU	S	*	*	*	*	Strobe for horiz sync with VB and EQU
DFF03A	STRVBL	S	*	*	*	*	Strobe for horiz sync with VB (vert blank)
DFF03C	STRHOR	S	*	*	*	*	Strobe for horiz sync
DFF03E	STRLONG	S	*	*	*	*	Strobe for identification of long horiz line
DFF040	BLTCON0	W	*	*	*	*	Blitter control register 0
DFF042	BLTCON1	W	*	*	*	*	Blitter control register 1
DFF044	BLTAFWM	W	*	*	*	*	Blitter first word mask for source A
DFF046	BLTALWM	W	*	*	*	*	Blitter last word mask for source A
DFF048	BLTCPTH	W	*	*	*	*	Blitter pointer to source C (high 5 bits, was 3 bits)

DFF04A	BLTCPTL	W	*	*	*	*	Blitter pointer to source C (low 15 bits)
DFF04C	BLTBPTH	W	*	*	*	*	Blitter pointer to source B (high 5 bits, was 3 bits)
DFF04E	BLTBPTL	W	*	*	*	*	Blitter pointer to source B (low 15 bits)
DFF050	BLTAPTH	W	*	*	*	*	Blitter pointer to source A (high 5 bits, was 3 bits)
DFF052	BLTAPTL	W	*	*	*	*	Blitter pointer to source A (low 15 bits)
DFF054	BLTDPTH	W	*	*	*	*	Blitter pointer to dest D (high 5 bits, was 3 bits)
DFF056	BLTDPTL	W	*	*	*	*	Blitter pointer to dest D (low 15 bits)
DFF058	BLTSIZE	W	*	*	*	*	Blitter start and size (win/width,height)
DFF05A	BLTCON0L	W		*	*	*	Blitter control 0, lower 8 bits (minterms)
DFF05C	BLTSIZV	W		*	*	*	Blitter V size (for 15 bit vertical size)
DFF05E	BLTSIZH	W		*	*	*	Blitter H size and start (for 11 bit H size)
DFF060	BLTCMOD	W	*	*	*	*	Blitter modulo for source C
DFF062	BLTBMOD	W	*	*	*	*	Blitter modulo for source B
DFF064	BLTAMOD	W	*	*	*	*	Blitter modulo for source A
DFF066	BLTDMOD	W	*	*	*	*	Blitter modulo for dest D
DFF070	BLTCDAT	W	*	*	*	*	Blitter source C data register
DFF072	BLTBDAT	W	*	*	*	*	Blitter source B data register
DFF074	BLTADAT	W	*	*	*	*	Blitter source A data register
DFF078	SPRHDT	W		*	*	*	Ext. logic UHRES sprite pointer and data identifier
DFF07A	BPLHDT	W		*	*	*	Ext. logic UHRES bit plane identifier
DFF07C	DENISEID	R		*	*	*	Chip revision level for Denise/Lisa (video out chip)
DFF07E	DSKSYNC	W	*	*	*	*	Disk sync pattern reg for disk read
DFF080	COP1LCH	W	*	*	*	*	Coprocessor 1st location (high 5 bits, was 3 bits)
DFF082	COP1LCL	W	*	*	*	*	Coprocessor 1st location (low 15 bits)
DFF084	COP2LCH	W	*	*	*	*	Coprocessor 2nd location (high 5 bits, was 3 bits)
DFF086	COP2LCL	W	*	*	*	*	Coprocessor 2nd location (low 15 bits)
DFF088	COPJMP1	S	*	*	*	*	Coprocessor restart at 1st location
DFF08A	COPJMP2	S	*	*	*	*	Coprocessor restart at 2nd location
DFF08C	COPINS	S	*	*	*	*	Coprocessor inst fetch identify
DFF08E	DIWSTRT	W	*	*	*	*	Display window start (upper left vert, horiz pos)
DFF090	DIWSTOP	W	*	*	*	*	Display window stop (lower right vert, horiz pos)
DFF092	DDFSTRT	W	*	*	*	*	Display bit plane data fetch start, horiz pos
DFF094	DDFSTOP	W	*	*	*	*	Display bit plane data fetch stop, horiz pos
DFF096	DMACON	W	*	*	*	*	DMA control write (clear or set)
DFF098	CLXCON	W	*	*	*	*	Collision control
DFF09A	INTENA	W	*	*	*	*	Interrupt enable bits (clear or set bits)
DFF09C	INTREQ	W	*	*	*	*	Interrupt request bits (clear or set bits)
DFF09E	ADKCON	W	*	*	*	*	Audio, disk, UART control
DFF0A0	AUD0LCH	W	*	*	*	*	Audio channel 0 location (high 5 bits was 3 bits)
DFF0A2	AUD0LCL	W	*	*	*	*	Audio channel 0 location (low 15 bits)
DFF0A4	AUD0LEN	W	*	*	*	*	Audio channel 0 length
DFF0A6	AUD0PER	W	*	*	*	*	Audio channel 0 period
DFF0A8	AUD0VOL	W	*	*	*	*	Audio channel 0 volume
DFF0AA	AUD0DAT	W	*	*	*	*	Audio channel 0 data
DFF0B0	AUD1LCH	W	*	*	*	*	Audio channel 1 location (high 5 bits was 3 bits)
DFF0B2	AUD1LCL	W	*	*	*	*	Audio channel 1 location (low 15 bits)
DFF0B4	AUD1LEN	W	*	*	*	*	Audio channel 1 length
DFF0B6	AUD1PER	W	*	*	*	*	Audio channel 1 period
DFF0B8	AUD1VOL	W	*	*	*	*	Audio channel 1 volume
DFF0BA	AUD1DAT	W	*	*	*	*	Audio channel 1 data
DFF0C0	AUD2LCH	W	*	*	*	*	Audio channel 2 location (high 5 bits was 3 bits)
DFF0C2	AUD2LCL	W	*	*	*	*	Audio channel 2 location (low 15 bits)
DFF0C4	AUD2LEN	W	*	*	*	*	Audio channel 2 length
DFF0C6	AUD2PER	W	*	*	*	*	Audio channel 2 period
DFF0C8	AUD2VOL	W	*	*	*	*	Audio channel 2 volume
DFF0CA	AUD2DAT	W	*	*	*	*	Audio channel 2 data
DFF0CC	AUD2VOL2	W				*	Audio channel 2 volume far side
DFF0D0	AUD3LCH	W	*	*	*	*	Audio channel 3 location (high 5 bits was 3 bits)
DFF0D2	AUD3LCL	W	*	*	*	*	Audio channel 3 location (low 15 bits)
DFF0D4	AUD3LEN	W	*	*	*	*	Audio channel 3 length
DFF0D6	AUD3PER	W	*	*	*	*	Audio channel 3 period
DFF0D8	AUD3VOL	W	*	*	*	*	Audio channel 3 volume
DFF0DA	AUD3DAT	W	*	*	*	*	Audio channel 3 data
DFF0E0	BPL1PTH	W	*	*	*	*	Bitplane pointer 1 (high 5 bits was 3 bits)
DFF0E2	BPL1PTL	W	*	*	*	*	Bitplane pointer 1 (low 15 bits)
DFF0E4	BPL2PTH	W	*	*	*	*	Bitplane pointer 2 (high 5 bits was 3 bits)
DFF0E6	BPL2PTL	W	*	*	*	*	Bitplane pointer 2 (low 15 bits)
DFF0E8	BPL3PTH	W	*	*	*	*	Bitplane pointer 3 (high 5 bits was 3 bits)
DFF0EA	BPL3PTL	W	*	*	*	*	Bitplane pointer 3 (low 15 bits)
DFF0EC	BPL4PTH	W	*	*	*	*	Bitplane pointer 4 (high 5 bits was 3 bits)

DFF0EE	BPL4PTL	W	*	*	*	*	Bitplane pointer 4 (low 15 bits)
DFF0F0	BPL5PTH	W	*	*	*	*	Bitplane pointer 5 (high 5 bits was 3 bits)
DFF0F2	BPL5PTL	W	*	*	*	*	Bitplane pointer 5 (low 15 bits)
DFF0F4	BPL6PTH	W	*	*	*	*	Bitplane pointer 6 (high 5 bits was 3 bits)
DFF0F6	BPL6PTL	W	*	*	*	*	Bitplane pointer 6 (low 15 bits)
DFF0F8	BPL7PTH	W			*	*	Bitplane pointer 7 (high 5 bits was 3 bits)
DFF0FA	BPL7PTL	W			*	*	Bitplane pointer 7 (low 15 bits)
DFF0FC	BPL8PTH	W			*	*	Bitplane pointer 8 (high 5 bits was 3 bits)
DFF0FE	BPL8PTL	W			*	*	Bitplane pointer 8 (low 15 bits)
DFF100	BPLCON0	W	*	*	*	*	Bitplane control (miscellaneous control bits)
DFF102	BPLCON1	W	*	*	*	*	Bitplane control (scroll value)
DFF104	BPLCON2	W	*	*	*	*	Bitplane control (video priority control)
DFF106	BPLCON3	W	*	*	*	*	Bitplane control (enhanced features)
DFF108	BPLMOD	W	*	*	*	*	Bitplane modulo (odd planes)
DFF10A	BPL2MOD	W	*	*	*	*	Bitplane modulo (even planes)
DFF10C	BPLCON4	W			*	*	Bitplane control (bitplane and sprite-masks)
DFF10E	CLXCON2	W			*	*	Extended collision control
DFF110	BPL1DAT	W	*	*	*	*	Bitplane 1 data (parallel to serial convert)
DFF112	BPL2DAT	W	*	*	*	*	Bitplane 2 data (parallel to serial convert)
DFF114	BPL3DAT	W	*	*	*	*	Bitplane 3 data (parallel to serial convert)
DFF116	BPL4DAT	W	*	*	*	*	Bitplane 4 data (parallel to serial convert)
DFF118	BPL5DAT	W	*	*	*	*	Bitplane 5 data (parallel to serial convert)
DFF11A	BPL6DAT	W	*	*	*	*	Bitplane 6 data (parallel to serial convert)
DFF11C	BPL7DAT	W			*	*	Bitplane 7 data (parallel to serial convert)
DFF11E	BPL8DAT	W			*	*	Bitplane 8 data (parallel to serial convert)
DFF120	SPROPPTH	W	*	*	*	*	Sprite 0 pointer (high 5 bits was 3 bits)
DFF122	SPROPPTL	W	*	*	*	*	Sprite 0 pointer (low 15 bits)
DFF124	SPR1PTH	W	*	*	*	*	Sprite 1 pointer (high 5 bits was 3 bits)
DFF126	SPR1PTL	W	*	*	*	*	Sprite 1 pointer (low 15 bits)
DFF128	SPR2PTH	W	*	*	*	*	Sprite 2 pointer (high 5 bits was 3 bits)
DFF12A	SPR2PTL	W	*	*	*	*	Sprite 2 pointer (low 15 bits)
DFF12C	SPR3PTH	W	*	*	*	*	Sprite 3 pointer (high 5 bits was 3 bits)
DFF12E	SPR3PTL	W	*	*	*	*	Sprite 3 pointer (low 15 bits)
DFF130	SPR4PTH	W	*	*	*	*	Sprite 4 pointer (high 5 bits was 3 bits)
DFF132	SPR4PTL	W	*	*	*	*	Sprite 4 pointer (low 15 bits)
DFF134	SPR5PTH	W	*	*	*	*	Sprite 5 pointer (high 5 bits was 3 bits)
DFF136	SPR5PTL	W	*	*	*	*	Sprite 5 pointer (low 15 bits)
DFF138	SPR6PTH	W	*	*	*	*	Sprite 6 pointer (high 5 bits was 3 bits)
DFF13A	SPR6PTL	W	*	*	*	*	Sprite 6 pointer (low 15 bits)
DFF13C	SPR7PTH	W	*	*	*	*	Sprite 7 pointer (high 5 bits was 3 bits)
DFF13E	SPR7PTL	W	*	*	*	*	Sprite 7 pointer (low 15 bits)
DFF140	SPROPOS	W	*	*	*	*	Sprite 0 vert,horiz start pos data
DFF142	SPROCTL	W	*	*	*	*	Sprite 0 position and control data
DFF144	SPRODATA	W	*	*	*	*	Sprite 0 image data register A
DFF146	SPRODATB	W	*	*	*	*	Sprite 0 image data register B
DFF148	SPR1POS	W	*	*	*	*	Sprite 1 vert,horiz start pos data
DFF14A	SPR1CTL	W	*	*	*	*	Sprite 1 position and control data
DFF14C	SPR1DATA	W	*	*	*	*	Sprite 1 image data register A
DFF14E	SPR1DATB	W	*	*	*	*	Sprite 1 image data register B
DFF150	SPR2POS	W	*	*	*	*	Sprite 2 vert,horiz start pos data
DFF152	SPR2CTL	W	*	*	*	*	Sprite 2 position and control data
DFF154	SPR2DATA	W	*	*	*	*	Sprite 2 image data register A
DFF156	SPR2DATB	W	*	*	*	*	Sprite 2 image data register B
DFF158	SPR3POS	W	*	*	*	*	Sprite 3 vert,horiz start pos data
DFF15A	SPR3CTL	W	*	*	*	*	Sprite 3 position and control data
DFF15C	SPR3DATA	W	*	*	*	*	Sprite 3 image data register A
DFF15E	SPR3DATB	W	*	*	*	*	Sprite 3 image data register B
DFF160	SPR4POS	W	*	*	*	*	Sprite 4 vert,horiz start pos data
DFF162	SPR4CTL	W	*	*	*	*	Sprite 4 position and control data
DFF164	SPR4DATA	W	*	*	*	*	Sprite 4 image data register A
DFF166	SPR4DATB	W	*	*	*	*	Sprite 4 image data register B
DFF168	SPR5POS	W	*	*	*	*	Sprite 5 vert,horiz start pos data
DFF16A	SPR5CTL	W	*	*	*	*	Sprite 5 position and control data
DFF16C	SPR5DATA	W	*	*	*	*	Sprite 5 image data register A
DFF16E	SPR5DATB	W	*	*	*	*	Sprite 5 image data register B
DFF170	SPR6POS	W	*	*	*	*	Sprite 6 vert,horiz start pos data
DFF172	SPR6CTL	W	*	*	*	*	Sprite 6 position and control data
DFF174	SPR6DATA	W	*	*	*	*	Sprite 6 image data register A
DFF176	SPR6DATB	W	*	*	*	*	Sprite 6 image data register B
DFF178	SPR7POS	W	*	*	*	*	Sprite 7 vert,horiz start pos data

DFF17A	SPR7CTL	W	*	*	*	*	Sprite 7 position and control data
DFF17C	SPR7DATA	W	*	*	*	*	Sprite 7 image data register A
DFF17E	SPR7DATB	W	*	*	*	*	Sprite 7 image data register B
DFF180	COLOR00	W	*	*	*	*	Color table 0
DFF182	COLOR01	W	*	*	*	*	Color table 1
DFF184	COLOR02	W	*	*	*	*	Color table 2
DFF186	COLOR03	W	*	*	*	*	Color table 3
DFF188	COLOR04	W	*	*	*	*	Color table 4
DFF18A	COLOR05	W	*	*	*	*	Color table 5
DFF18C	COLOR06	W	*	*	*	*	Color table 6
DFF18E	COLOR07	W	*	*	*	*	Color table 7
DFF190	COLOR08	W	*	*	*	*	Color table 8
DFF192	COLOR09	W	*	*	*	*	Color table 9
DFF194	COLOR10	W	*	*	*	*	Color table 10
DFF196	COLOR11	W	*	*	*	*	Color table 11
DFF198	COLOR12	W	*	*	*	*	Color table 12
DFF19A	COLOR13	W	*	*	*	*	Color table 13
DFF19C	COLOR14	W	*	*	*	*	Color table 14
DFF19E	COLOR15	W	*	*	*	*	Color table 15
DFF1A0	COLOR16	W	*	*	*	*	Color table 16
DFF1A2	COLOR17	W	*	*	*	*	Color table 17
DFF1A4	COLOR18	W	*	*	*	*	Color table 18
DFF1A6	COLOR19	W	*	*	*	*	Color table 19
DFF1A8	COLOR20	W	*	*	*	*	Color table 20
DFF1AA	COLOR21	W	*	*	*	*	Color table 21
DFF1AC	COLOR22	W	*	*	*	*	Color table 22
DFF1AE	COLOR23	W	*	*	*	*	Color table 23
DFF1B0	COLOR24	W	*	*	*	*	Color table 24
DFF1B2	COLOR25	W	*	*	*	*	Color table 25
DFF1B4	COLOR26	W	*	*	*	*	Color table 26
DFF1B6	COLOR27	W	*	*	*	*	Color table 27
DFF1B8	COLOR28	W	*	*	*	*	Color table 28
DFF1BA	COLOR29	W	*	*	*	*	Color table 29
DFF1BC	COLOR30	W	*	*	*	*	Color table 30
DFF1BE	COLOR31	W	*	*	*	*	Color table 31
DFF1C0	HTOTAL	W	*	*	*	*	Highest number count, horiz line (VARBEAMEN=1)
DFF1C2	HSSTOP	W	*	*	*	*	Horizontal line position for HSYNC stop
DFF1C4	HBSTRT	W	*	*	*	*	Horizontal line position for HBLANK start
DFF1C6	HBSTOP	W	*	*	*	*	Horizontal line position for HBLANK stop
DFF1C8	VTOTAL	W	*	*	*	*	Highest numbered vertical line (VARBEAMEN=1)
DFF1CA	VSSSTOP	W	*	*	*	*	Vertical line position for VSYNC stop
DFF1CC	VBSTRT	W	*	*	*	*	Vertical line for VBLANK start
DFF1CE	VBSTOP	W	*	*	*	*	Vertical line for VBLANK stop
DFF1D0	SPRHSTRT	W	*	*	*	*	UHRES sprite vertical start
DFF1D2	SPRHSTOP	W	*	*	*	*	UHRES sprite vertical stop
DFF1D4	BPLHSTRT	W	*	*	*	*	UHRES bit plane vertical start
DFF1D6	BPLHSTOP	W	*	*	*	*	UHRES bit plane vertical stop
DFF1D8	HHPOSW	W	*	*	*	*	DUAL mode hires H beam counter write
DFF1DA	HHPOSR	R	*	*	*	*	DUAL mode hires H beam counter read
DFF1DC	BEAMCON0	W	*	*	*	*	Beam counter control register (SHRES,UHRES,PAL)
DFF1DE	HSSTRT	W	*	*	*	*	Horizontal sync start (VARHSY)
DFF1E0	VSSSTRT	W	*	*	*	*	Vertical sync start (VARVSY)
DFF1E2	HCENTER	W	*	*	*	*	Horizontal position for Vsync on interlace
DFF1E4	DIWHIGH	W	*	*	*	*	Display window - upper bits for start/stop
DFF1E6	BPLHMOD	W					Chunky plane modulo
DFF1E8	SPRHPTH	W	*	*	*	*	UHRES sprite pointer (high 5 bits)
DFF1EA	SPRHPTL	W	*	*	*	*	UHRES sprite pointer (low 15 bits)
DFF1EC	BPLHPTH	W				*	Chunky plane pointer (hi 15 bits)
DFF1EE	BPLHPTL	W				*	Chunky plane pointer (lo 15 bits)
DFF1F4	GFXMODE	W				*	Set Resolution and Pixformat
DFF1FC	FMODE	W		*	*		Fetch mode register
Address	Name	Access	OCS	ECS	AGA	SAGA	Description
DFF202	DMACON2	Read	R		*		
DFF21c	INTENA2	Read	R		*		
DFF21E	INTREQ2	Read	R		*		
DFF220	joy0buttons	R			*		16bit USB Joy Buttons (0)=active
DFF222	joy1buttons	R			*		16bit USB Joy Buttons (0)=active
DFF224	joy2buttons	R			*		16bit USB Joy Buttons (0)=active

DFF226	joy3buttons	R		* 16bit USB Joy Buttons (0)=active
DFF230	clxd0	R		* Sprite0 Detailed Collision
DFF232	clxd1	R		* Sprite1 Detailed Collision
DFF234	clxd2	R		* Sprite2 Detailed Collision
DFF236	clxd3	R		* Sprite3 Detailed Collision
DFF238	clxd4	R		* Sprite4 Detailed Collision
DFF23A	clxd5	R		* Sprite5 Detailed Collision
DFF23C	clxd6	R		* Sprite6 Detailed Collision
DFF23E	clxd7	R		* Sprite7 Detailed Collision
DFF296	DMACON2	W		* DMACON2
DFF29A	INTENA2	W		* INTENA2
DFF29C	INTREQ2	W		* INTREQ2
DFF380	PlanarCOLH	W		* 32bit Planar COLOR Port register
DFF382	PlanarCOLL	W		* 256 Color Register, Format [ID,RR,GG,BB]
DFF384	SpriteCOLH	W		* 32bit Sprite COLOR Port register
DFF386	SpriteCOLL	W		* 256 Color Register, Format [ID,RR,GG,BB]
DFF388	ChunkyCOLH	W		* 32bit Chunky COLOR Port register
DFF38A	ChunkyCOLL	W		* 256 Color Register, Format [ID,RR,GG,BB]
DFF38C	PIPCOLH	W		* 32bit PIP COLOR Port register
DFF38E	PIPCOLL	W		* 256 Color Register, Format [ID,RR,GG,BB]
DFF3D0	PipXStrt	W		* X start
DFF3D2	PipYStrt	W		* Y start
DFF3D4	PipXStop	W		* X stop
DFF3D6	PipYStop	W		* Y stop
DFF3D8	PipPtr	W		* 32 bit Ptr
DFF3DC	PipFormat	W		* Colorformat (8bit/16bit/15bit/YUV)
DFF3DE	PipModulo	W		* Modulo
DFF3E0	PipColorKey	W		* ColorKey 1000,RRRR,GGGG,BBBB
DFF3E2	PipDMARowLen	W		* PipDMARowLen
DFF3FC	VAMPIRE VERSION	R		* 8bit Card Version / 8bit clock multiplier

Address	Name	Access	OCS	ECS	AGA	SAGA	Description
DFF400	AUD0L	W	*				Audio channel 0 location (32bit PTR)
DFF402	AUD0L	W	*				
DFF404	AUD0LEN	W		*			Audio channel 0 length (32bit)
DFF406	AUD0LEN	W		*			
DFF408	AUD0VOL	W		*			2x8bit VOL (LEFT/RIGHT)
DFF40A	AUD0CTRL	W		*			16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF40C	AUD0PER	W		*			16bit PERIOD
DFF410	AUD1L	W	*				Audio channel 1 location (32bit PTR)
DFF412	AUD1L	W	*				
DFF414	AUD1LEN	W		*			Audio channel 1 length (32bit)
DFF416	AUD1LEN	W		*			
DFF418	AUD1VOL	W		*			2x8bit VOL (LEFT/RIGHT)
DFF41A	AUD1CTRL	W		*			16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF41C	AUD1PER	W		*			16bit PERIOD
DFF420	AUD2L	W	*				Audio channel 2 location (32bit PTR)
DFF422	AUD2L	W	*				
DFF424	AUD2LEN	W		*			Audio channel 2 length (32bit)
DFF426	AUD2LEN	W		*			
DFF428	AUD2VOL	W		*			2x8bit VOL (LEFT/RIGHT)
DFF42A	AUD2CTRL	W		*			16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF42C	AUD2PER	W		*			16bit PERIOD
DFF430	AUD3L	W	*				Audio channel 3 location (32bit PTR)
DFF432	AUD3L	W	*				
DFF434	AUD3LEN	W		*			Audio channel 3 length (32bit)
DFF436	AUD3LEN	W		*			
DFF438	AUD3VOL	W		*			2x8bit VOL (LEFT/RIGHT)
DFF43A	AUD3CTRL	W		*			16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF43C	AUD3PER	W		*			16bit PERIOD
DFF440	AUD4L	W	*				Audio channel 4 location (32bit PTR)
DFF442	AUD4L	W	*				
DFF444	AUD4LEN	W		*			Audio channel 4 length (32bit)
DFF446	AUD4LEN	W		*			
DFF448	AUD4VOL	W		*			2x8bit VOL (LEFT/RIGHT)
DFF44A	AUD4CTRL	W		*			16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF44C	AUD4PER	W		*			16bit PERIOD
DFF450	AUD5L	W	*				Audio channel 5 location (32bit PTR)
DFF452	AUD5L	W	*				
DFF454	AUD5LEN	W		*			Audio channel 5 length (32bit)

DFF456	AUD5LEN	W	*
DFF458	AUD5VOL	W	* 2x8bit VOL (LEFT/RIGHT)
DFF45A	AUD5CTRL	W	* 16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF45C	AUD5PER	W	* 16bit PERIOD
DFF460	AUD6L	W	* Audio channel 6 location (32bit PTR)
DFF462	AUD6L	W	*
DFF464	AUD6LEN	W	* Audio channel 6 length (32bit)
DFF466	AUD6LEN	W	*
DFF468	AUD6VOL	W	* 2x8bit VOL (LEFT/RIGHT)
DFF46A	AUD6CTRL	W	* 16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF46C	AUD6PER	W	* 16bit PERIOD
DFF470	AUD7L	W	* Audio channel 7 location (32bit PTR)
DFF472	AUD7L	W	*
DFF474	AUD7LEN	W	* Audio channel 7 length (32bit)
DFF476	AUD7LEN	W	*
DFF478	AUD7VOL	W	* 2x8bit VOL (LEFT/RIGHT)
DFF47A	AUD7CTRL	W	* 16 Control bits 2= Stereo 1= Oneshot 0= (8bit/16bit)
DFF47C	AUD7PER	W	* 16bit PERIOD

Joystick Read

JOYBUTTONS

This register contains extra fire buttons (USB joypads)

BIT#	FUNCTION	DESCRIPTION
15	Right	Move RIGHT
14	Left	Move Left
13	Down	Move Down
12	Up	Move Up
11	Reserved	
10	Start	Start
09	Back	Back
08	FIRE8	Firebutton 8
07	FIRE7	Firebutton 7
06	FIRE6	Firebutton 6
05	FIRE5	Firebutton 5
04	FIRE4	Firebutton 4
03	FIRE3	Firebutton 3
02	FIRE2	Firebutton 2
01	FIRE1	Firebutton 1
00	PLUG	Joypad connected

Graphics Modes

GFXMODE

16bit register selecting the screen size and the pixelformat for the Chunky plane.

BIT#	FUNCTION	DESCRIPTION
15.08	Resolution	\$00
		\$01 = 320x200
		\$02 = 320x240
		\$03 = 320x256
		\$04 = 640x400
		\$05 = 640x480
		\$06 = 640x512

		\$07 = 960x540	
		\$08 = 480x270	
		\$09 = 304x224	
		\$0A = 1280x720	
		\$0B = 640x360	
		\$0C = 800x600	
		\$0D = 1024x768	
		\$0E = 720x576	
		\$0F = 848x480	
+-----+			
07.00	Pixelformat	\$00	
		\$01 = 8bit (indexed)	
		\$02 = 16bit R5G6B5	
		\$03 = 15bit 1R5G5B5	
		\$04 = 24bit R8G8B8	
		\$05 = 32bit A8R8G8B8	
		\$06 = YUV	
		\$07 =	
		\$08 = PLANAR 1BIT	
		\$09 = PLANAR 2BIT	
		\$0A = PLANAR 4BIT	
+-----+			

Registers

INTEGER UNIT USER PROGRAMMING MODEL

16 64-Bit Address registers (A0-A15)

32 64-Bit General Purpose Data registers (D0-D7,E0-E23)

8 FPU registers (Fp0-Fp7,)

32 Data Registers (D0-D7,E0-E23)

These registers are for bit and bit field (1 - 32 bits), byte (8 bits), word (16 bits), long-word (32 bits), and quad-word (64 bits) operations. D0-D7 can also be used as index registers.

8 FPU Registers (Fp0-Fp7,E0-E23)

The FPU has own 8 registers. But FPU instructions can also use register EN as destination and EN and Dn as source. Therefore the FPU has 32 registers it can update, and 40 registers it can use as source.

16 Address Registers (A0-A15)

These registers can be used as software stack pointers, index registers, or base address registers. The base address registers can be used for word and long-word operations. Register A7 is used as a hardware stack pointer during stacking for subroutine calls and exception handling. In the user programming model, A7 refers to the user stack pointer (USP).

SPECIAL PURPOSE REGISTERS

000 SFC	00C STALLS HAZARD
001 DFC	00D STALLS BUFFER
002 CACR	00E MEMORY WRITES
003 MMU TC	00F ERROR_PC
004 ITT0	010 FPUInputA
005 ITT1	801 VBR
006 DTT0	808 PCR
007 DTT1	809 CPU CLOCKS
008 BUSCTRL	80A MIPS Pipe 1
009 BREAKPOINT	80B MIPS Pipe 2
00A STALLS REGISTER	80C BPC Branches Predicted Correct
00B STALLS CACHE	80D BPW Branches Predicted Wrong

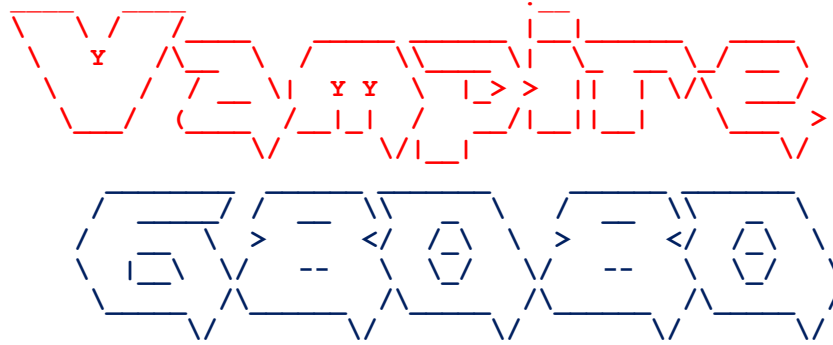
80E Data Cache Hits
80F Data Cache Miss

810 FEMU BASE

Vampire Quick Links

ApolloS Cores & OS	http://www.apollo-core.com/downloads/
Graphics Modes	http://www.apollo-core.com/sagadoc/gfxmode.htm
SAGA Information	http://www.apollo-core.com/sagadoc/
Certified Monitors	https://docs.google.com/spreadsheets/d/1iCv5PGaA2yW9T0sqFOix_vTfOgrHoQ2Op-60f9Me1Pc/edit#gid=0
WHDLoad Files	https://docs.google.com/spreadsheets/d/1w01R98NkMpevUox1N2iFS3-090jexIJqG11N2dC5AeM/edit#gid=504929201
AMMX Information	http://www.apollo-core.com/AMMX.doc.txt http://www.apollo-core.com/instructions.htm?b=4&z=YnFmNH
68080 Instructions	http://www.apollo-core.com/index.htm?page=instructions
Vampire Bootloader	https://drive.google.com/drive/folders/1aCw5hQWd24WclqHRl22XZr6vuotjtxkl
Cotter's Beta Stuff	https://drive.google.com/drive/folders/1TsafNzgYVh45JIzW7alxhuIChPBacRiD
Coffin OS	http://coffin.ninex.info/

Compiled from www.apollo-core.com & Motorola's 1994 M68060 User's Manual



Summary of Instructions

Mnemonic	Description	Syntax
ABCD	Add Decimal With Extend	ABCD Dy, Dx ABCD -(Ay), -(Ax)
ADD	Add	ADD <ea>, Dn ADD Dn, <ea>
ADDA	Add Address	ADDA <ea>, An
ADDI	Add Immediate	ADDI #<data>, <ea>
ADDQ	Add Quick	ADDQ #<data>, An
ADDX	Add With Extend	ADDX Dy, Dx ADDX -(Ay), -(Ax)
AND	Logical AND	AND <EA>, DN AND DN, <EA>
ANDI	Logical AND Immediate	ANDI #<data>, <ea>
ASL, ASR	Arithmetic Shift Left And Right	ASx Dx, Dy ASx #<data>, Dy ASx <ea>
Bcc	Branch Conditionally	Bcc <label>
BCHG	Test Bit And Change	BCHG Dn, <ea> BCHG #<data>, <ea>
BCLR	Test Bit And Clear	BCLR Dn, <ea> BCLR #<data>, <ea>
BFCHG	Test Bit Field And Change	BFCHG <ea>{offset:width}
BFCLR	Test Bit Field And Clear	BFCLR <ea>{offset:width}
BFEXTS	Signed Bit Field Extract	BFEXTS <ea>{offset:width}, Dn
BFEXTU	Unsigned Bit Field Extract	BFEXTU <ea>{offset:width}, Dn
BFFFO	Bit Field Find First One	BFFFO <ea>{offset:width}, Dn
BFINS	Bit Field Insert	BFINS Dn, <ea>{offset:width}
BFSET	Test Bit Field And Set	BFSET <ea>{offset:width}
BFTST	Test Bit Field	BFTST <ea>{offset:width}
BKPT	Breakpoint	BKPT #<data>
BRA	Branch	BRA <label>
BSEL	Performs the bitwise operation	BSEL <EA>, DN,DM
BSET	Test Bit And Set	BSET Dn, <ea> BSET #<data>, <ea>
BSR	Branch To Subroutine	BSR <label>
BTST	Test Bit	BTST Dn, <ea> BTST #<data>, <ea>
CAS	Compare And Swap Operands	CAS Dc, Du, <ea>
CAS2	Compare And Swap Dual Operands	CAS2 Dc1:Dc2, Du1:Du2, (Rn1):(Rn2)
CHK	Check Register Against Bounds	CHK <ea>, Dn

Mnemonic	Description	Syntax
CHK2	Check Register Against Upper And Lower Bounds	CHK2 <ea>, Rn
CINV	Invalidate Cache Entries	
CLR	Clear	
CMP	Compare	CMP <ea>, Dn
CMPA	Compare Address	CMPA <ea>, An
CMPI	Compare Immediate	CMPI #<data>, <ea>
CMPM	Compare Memory To Memory	CMPM (Ay)+, (Ax)+
CMP2	Compare Register Against Upper And Lower Bounds	CMP2 <ea>, Rn
CPUSH	Push Then Invalidate Cache Entries	

DBCC	Test Condition, Decrement And Branch	DBcc Dn,<label>
DIVS, DIVSL	Signed Divide	DIVS.W <ea>, Dn 32/16 -->; 16r:16q DIVS.L <ea>, Dq 32/32 -->; 32q DIVS.L <ea>, Dr:Dq 64/32 -->; 32r:32q DIVSL.L <ea>, Dr:Dq 32/32 -->; 32r:32q
DIVU, DIVUL	Unsigned Divide	DIVU.W <ea>, Dn 32/16 -->; 16r:16q DIVU.L <ea>, Dq 32/32 -->; 32q DIVU.L <ea>, Dr:Dq 64/32 -->; 32r:32q DIVUL.L <ea>, Dr:Dq 32/32 -->; 32r:32q

EOR	Logical Exclusive OR	EOR Dn, <ea>
EORI	Logical Exclusive OR Immediate	EORI #<data>, <ea>
EORI to CCR	Exclusive OR Immediate to the Condition Code Register	EORI #<data>, CCR
EXG	Exchange Registers	Rx \s--->; Ry
EXT, EXTB	Sign Extend	EXT.W Dn extends byte to word EXT.L Dn extends word to long word EXTB.L Dn extends byte to long word

ILLEGAL	Take Illegal Instruction Trap	ILLEGAL
---------	-------------------------------	---------

JMP	Jump	JMP <ea>
JSR	Jump To Subroutine	JSR <ea>

LEA	Load Effective Address	LEA <ea>,An
LINK	Link And Allocate	LINK An, #<displacement>
LSL, LSR	Logical Shift Left And Right	LSx Dx, Dy LSx #<data>, Dy LSx <ea>

MOVE	Move	MOVE <ea>, <ea>
MOVE16	16-byte Block Move	MOVE16 (Ax)+, (Ay)+ MOVE16 xxx.L, (An) MOVE16 xxx.L, (An)+ MOVE16 (An), xxx.L MOVE16 (An)+, xxx.L
MOVEA	Move Address	MOVEA <ea>, AN
MOVE CCR	Move Condition Code Register	MOVE CCR, <ea> MOVE <ea>,CCR
MOVE SR	Move Status Register	MOVE SR, <ea> MOVE <ea>,SR
MOVE USP	Move User Stack Pointer	MOVE USP,An MOVE An,USP
MOVEC	Move Control Register	MOVEC CR, DN

Mnemonic	Description	Syntax
MOVEM	Move Peripheral	MOVEM register list, <ea> MOVEM <ea>, register list
MOVEP	Move Quick	MOVEP Dx, d(Ay) MOVEP d(Ay), Dx
MOVEQ	Move Alternate Address Space	
MOVES	Move Multiply	MOVES <ea>, RN MOVES Rn,<ea>
MOVEX		MOVEX <ea>, RN MOVEX Rn,<ea>
MULS	Signed Multiply	MULS.W <ea>, Dn 16 x 16 -->; 32 MULS.L <ea>, D1 32 x 32 -->; 32 MULS.L <ea>, Dh:D1 32 x 32 -->; 64
MULU	Unsigned Multiply	MULU.W <ea>, Dn 16 x 16 -->; 32 MULU.L <ea>, D1 32 x 32 -->; 32 MULU.L <ea>, Dh:D1 32 x 32 -->; 64

NBCD	Negate decimal with extend	NBCD <ea>
NEG	Negate	NEG <ea>
NEGX	Negate with extend	NEGX <ea>
NOP	No operation	NOP
NOT	Logical complement	NOT <ea>

OR	Logical Inclusive OR	OR <ea>, Dn OR Dn, <ea>
ORI	Logical Inclusive OR Immediate	ORI #<data>, <ea>

PACK	Pack BCD	PACK -(Ax), -(Ay), #<adjustment> PACK Dx, Dy, #<adjustment>
PEA	Push Effective Address	PEA <ea>
PERM	Performs a byte permutation of two data registers	PERM <#imm>, Dn,Dm
PFLUSH	Flush Entry(ies) In The ATCs	
PTEST	Test A Logical Address	

RESET	Reset External Devices	
ROL, ROR	Rotate Left And Right	ROx Dx, Dy ROx #<data>, Dy ROx* <ea>
ROXL, ROXR	Rotate With Extend Left And Right	ROXx Dx, Dy ROXx #<data>, Dy ROXx* <ea>
RPC	Return Program Counter	
RTD	Return And Deallocate	RTD #<displacement>
RTE	Return From Exception	RTE
RTR	Return And Restore Codes	RTR
RTS	Return From Subroutine	RTS

SB CD	Subtract Decimal With Extend	SB CD Dx, Dy SB CD -(Ax), -(Ay)
Scc	Set Conditionally	Scc <label>
STOP	Stop	STOP #<data>
SUB	Subtract	SUB <ea>, Dn SUB Dn, <ea>
SUBA	Subtract Address	SUBA <ea>, An
SUBI	Subtract Immediate	SUBI #<data>, <ea>
SUBQ	Subtract Quick	SUBQ #<data>, An;
SUBX	Subtract With Extend	SUBX Dx, Dy
SWAP	Swap Register Words	SWAP Dn

Mnemonic	Description	Syntax
TAS	Test Operand And Set	TAS <ea>
TRAP	Trap	TRAP #<vector>
TRAPcc	Trap Conditionally	TRAPcc
TRAPV	Trap On Overflow	TRAPV
TST	Trap Operand	TST <ea>

UNLK	Unlink	UNLK An
UNPK	Unpack BCD	UNPK -(Ax), -(Ay), #<adjustment> UNPK Dx, Dy, #<adjustment>

Floating-Point Instructions

Mnemonic	Description	Syntax
FABS	Floating-point Absolute Value	FABS.<fmt> <ea>,FPn FABS.X FPm,FPn FABS.X FPn FrABS.<fmt> <ea>,FPn3 FrABS.X FPm,FPn3 FrABS.X FPn3
FADD	Floating-point Add	FADD.<fmt> <ea>,FPn ; FADD.X FPm,FPn ; FrADD.<fmt> <ea>,FPn3 ; FrADD.X FPm,FPn3
FBcc	Branch On Floating-point Condition	
FCMP	Floating-point Compare	
FDBcc	Floating-point Decrement And Branch	
FDIV	Floating-point Divide	FDIV.<fmt> <ea>,FPn FDIV.X FPm,FPn FrDIV.<fmt> <ea>,FPn3 ; FrDIV.X FPm,FPn3
FMOVE	Move Floating-point Register	FMOVE.<fmt> <ea>,FPn FMOVE.<fmt> FPm,<ea> FMOVE.P FPm,<ea>{Dn} FMOVE.P FPm,<ea>{#k} FrMOVE.<fmt> <ea>,FPn3
FMOVEM	Move Multiple Floating-point Registers	
FMUL	Floating-point Multiply	FMUL.<fmt> <ea>,FPn FMUL.X FPm,FPn FrMUL.<fmt> <ea>,FPn3 FrMUL.X FPm,FPn3
FNEG	Floating-point Negate	FNEG.<fmt> <ea>,FPn FNEG.X FPm,FPn FNEG.X FPn FrNEG.<fmt> <ea>,FPn3 FrNEG.X FPm,FPn3 FrNEG.X FPn3
FRESTORE	Restore Floating-point Internal State	FRESTORE <ea>
FSAVE	Save Floating-point Internal State	FSAVE <ea>

Mnemonic	Description	Syntax
FScc	Set According To Floating-point Condition	
FSQRT	Floating-point Square Root	
FSUB	Floating-point Substract	FSUB.<fmt> <ea>,FPn FSUB.X FPM,FPn FrSUB.<fmt> <ea>,FPn3 FrSUB.X FPM,FPn3
FTRAPcc	Trap On Floating-point Condition	
FTST	Floating-point Test	

AMMX Instructions

Mnemonic	Description	Syntax
BANK	Register Bank Switching	BANK SrcA,SrcB,Size
BFLYW	butterfly operation, single-cycle vector short addition	bflyw <VEA>,b,d:d+1
BSEL	bit-by-bit selection	bsel <VEA>,b,d

C2P	chunky to planar conversion	c2p <VEA>,d
-----	-----------------------------	-------------

LOAD	load 64 bit into destination register	load <VEA>,d
LOADI	load 64 bit indirect into destination register	loadi <VEA>,d
LSLQ	64 Bit shift left	lslq <VEA>,b,d
LSRQ	64 Bit shift right	lsrq <VEA>,b,d

PABS	Each element is converted to absolute value = positiv	PABSB, PABSW
PACKUSWB	Pack 2x4 signed shorts into 8 unsigned char, saturate to 0..255	packuswb a,b,<VEA>
PACK3216	pack 32 Bit ARGB data into 16 Bit RGB565	pack3216 a,b,<VEA>
PADDB	vector add bytes	paddb <VEA>,b,d
PADDUSB	vector add unsigned bytes with unsigned saturation:	paddusb <VEA>,b,d
PADDUSW	vector add unsigned short with saturation	paddusw <VEA>,b,d
PADDW	vector add short:	paddw <VEA>,b,d
PAND POR PEORPANDN	64 Bit logic operations	pand <VEA>,b,d por <VEA>,b,d peor <VEA>,b,d pandn <VEA>,b,d
PAVGB	average 8 unsigned bytes with 8 unsigned bytes	pavgb <VEA>,b,d
PCMPccb	byte-by-byte vector compare	pcmpeq b <VEA>,b,d pcmpgt b <VEA>,b,d pcmpge b <VEA>,b,d pcmphi b <VEA>,b,d
PCMPccw	short-by-short vector compare	pcmpeq w <VEA>,b,d pcmpgt w <VEA>,b,d pcmpge w <VEA>,b,d pcmphi w <VEA>,b,d
PMAXxB	byte-by-byte vector compare and obtain larger	pmaxub <VEA>,b,d pmaxsb <VEA>,b,d

Mnemonic	Description	Syntax
PMAXxW	short-by-short vector compare and obtain larger	pmaxuw <VEA>,b,d pmaxsw <VEA>,b,d
PMINxB	byte-by-byte vector compare and obtain smaller	pminub <VEA>,b,d pminsb <VEA>,b,d
PMINxW	short-by-short vector compare and obtain smaller	pminuw <VEA>,b,d pminsw <VEA>,b,d
PMUL88		
PMULA	vector multiply and add unsigned bytes	pmula <VEA>,b,d
PMULH	vector multiply short and keep upper 16 Bit	pmulh <VEA>,b,d

PMULL	vector multiply short and keep lower 16 Bit	pmull <VEA>,b,d
PMULL88	vector multiply short and shift down 8	pmul88 <VEA>,b,d
PSUBB	vector subtract bytes	psubb <VEA>,b,d
PSUBUSB	vector subtract unsigned bytes with unsigned saturation	psubusb <VEA>,b,d
PSUBUSW		
PSUBW	vector subtract short:	psubw <VEA>,b,d

STORE	store 64 bit from source register in memory	store a,<VEA>
STOREC	store at most "count" bytes from a into destination	storec a,count,<VEA>
STOREi	store 64 bit indirect source into memory location	storei a,<VEA>
STOREilm	store bytes from a into destination, based on inverted long mask	storeilm a,m,<VEA>
STOREm	store a selection of bytes from a into	storem a,m,<VEA>

TRANSHI	matrix transposition, upper half	transhi a-d , e:f
TRANSLO	matrix transposition, lower half	translo a-d , e:f

UNPACK1632	unpack 16 Bit RGB565 data into 32 Bit ARGB	unpack1632 <VEA>,d:d+1
------------	--	------------------------

VPERM	permute the contents of two registers into destination register	vperm #N,a,b,d
-------	---	----------------

Compiled by:

DARRIN

Instruction Timings

Due to the speed of the 68080 most instructions will require less than a cycle to execute.

You can normally execute 2 instructions per cycle if the instructions are not dependent on one another. These work well together since the 2nd instruction is not dependent of the result of the first.

```
add.l #1,D5  
add.l #2,D6
```

The CPU can normally only execute non-dependent instructions together. There is an exception. Some bundles, like the one below, can be executed together. This is hard to explain, but keep it in mind.

```
move.l D0,D2  
add.l D1,D2  
move.l D5,D6  
sub.l D7,D6
```
